TECHNICAL REPORT · WAVEFORMX

Six waveform models in JAX and MLX, checked against LALSimulation

How TaylorF2, IMRPhenomD, IMRPhenomXAS, IMRPhenomXHM, the precessing IMRPhenomXPNR and the surrogate NRSur7dq4, and LALSimulation's conversions between the time and frequency domains, were written as pure functions that compile, vectorise and differentiate, in JAX on the CPU and in MLX on Apple GPUs, in float64 and float32. The report covers how they compare with LALSimulation 6.2.0, with waveformrs and with each other, how their derivatives compare with finite differences, and what they cost.

REPOSITORY	REFERENCE	PLATFORM	GENERATED
`waveformx`, branch `claude/xpnr-long-fast-d177a9` at `6f4c58d` (164 commits)	lalsimulation 6.2.0, lal 7.7.0 (conda-forge, osx-arm64), and waveformrs's Python bindings	Apple M2 Ultra, 24 cores; JAX 0.11.2 on the CPU, MLX on the Apple GPU; Python 3.12.15	25 October 2026

CONTENTS

1 Summary
2 Models as pure JAX functions
3 IMRPhenomXPNR
4 IMRPhenomXAS
5 IMRPhenomXHM
6 TaylorF2 and IMRPhenomD
7 NRSur7dq4
8 Time and frequency domains
9 How the package is tested
10 Accuracy
11 IMRPhenomXPNR against NRSur7dq4
12 Derivatives
13 Performance
14 Differences from LALSimulation
15 Limitations and future work
16 Reproducing the results
17 Commit history

1 Summary

waveformx implements six models in JAX: the frequency-domain models TaylorF2, IMRPhenomD, IMRPhenomXAS, IMRPhenomXHM and the precessing IMRPhenomXPNR, and the time-domain surrogate NRSur7dq4, with the interface of waveformrs's Python bindings and LALSimulation's conventions. Every model is a pure function of a `Binary` pytree, so it can be compiled with `jax.jit`, evaluated for many binaries at once with `jax.vmap`, and differentiated with respect to all twelve parameters of the binary with `jax.grad`, `jax.jvp` and `jax.jacfwd`. LALSimulation's `SimInspiralTD` and `SimInspiralFD` are reproduced as well, with LAL's lengths and start times, and are differentiable too. NRSur7dq4 follows LAL's code branch for branch, with each branch written as traced arithmetic, and agrees with LALSimulation 6.2.0 to rounding. IMRPhenomXPNR keeps LAL's physics but not its numerical procedure: it uses the SpinTaylor precession angles for every binary, solved once with a high-order integrator, with LAL's discrete switches made continuous and the parity fix of lalsuite !2593. Where LAL uses the SpinTaylor angles the two agree to small mismatches; the version that reproduced LAL's numerics step for step is commit `0bbd3c5`, tag `lal-faithful` (section 3). Every model also runs in MLX on Apple GPUs, every model can take its reference frequency scaled by the total mass (`reference_mf`), and no model computes in SI units.

Open items. The tests, the accuracy comparisons, the derivative checks and the timings below are complete. Still open:

- why finite differences of the full IMRPhenomXPNR waveform disagree with its derivatives in the in-plane spins (the derivatives section)
- JAX on one core: setting XLA's thread flags (`--xla_cpu_multi_thread_eigen=false intra_op_parallelism_threads=1`) did not change the ~3 cores it keeps busy, so the per-core comparison with LAL and waveformrs is still to be made
- the timings of IMRPhenomD on an idle machine: they were measured while other jobs ran (`report/taylorf2_phenomd.py`; the TaylorF2 and IMRPhenomD section)
- NRSur7dq4's float32 timings on an idle machine: they were measured while other jobs ran (`report/float32_nrsur7dq4.py --only-timings`; the NRSur7dq4 section)

IMRPHENOMXPNR AGAINST LAL 6.2.0 + !2593

≤ 1.7×10^{-4}

Largest mismatch over the 193 random binaries for which LAL keeps the SpinTaylor angles (median 2.8×10^{-8}); where LAL leaves them (7 binaries), up to 2.1×10^{-1}.

NRSUR7DQ4 AGAINST LAL 6.2.0

≤ 2.8×10^{-12}

Over 200 random binaries, with LAL's length and epoch in 200 of them; largest mismatch 3.3×10^{-16}.

NRSUR7DQ4 DERIVATIVES

≤ 2.9×10^{-8}

Largest disagreement of any column of the Jacobian with finite differences, over four binaries, wherever the waveform is differentiable; for IMRPhenomXPNR, 1.9×10^{-7} (section 12).

ONE IMRPHENOMXPNR WAVEFORM

7.81 ms

Compiled, on 8033 frequencies, against 13.82 ms for LALSimulation and 3.16 ms for waveformrs. The first call compiles for 13.9 s.

BATCHES WITH `JAX.VMAP`

204 / s

IMRPhenomXPNR waveforms per second at best over batch sizes, against 4,483 / s for waveformrs on all threads and 71 / s for LALSimulation on one.

TESTS

816 / 818 pass

pytest, including the slow random-draw tests, in 33 minutes. They compare the models with LALSimulation, waveformrs and its reference data.

COMPARISON	DRAWS	MAX REL. ERROR	MEDIAN REL. ERROR	MAX MISMATCH	MEDIAN MISMATCH
IMRPhenomXPNR, where LAL keeps the SpinTaylor angles, against 6.2.0 + lalsuite !2593 (waveformrs)	193/200	9.7×10^{-3}	5.0×10^{-5}	1.7×10^{-4}	2.8×10^{-8}
IMRPhenomXPNR, where LAL leaves them, against 6.2.0 + lalsuite !2593 (waveformrs)	7/200	5.4×10^{-1}	6.0×10^{-2}	2.1×10^{-1}	1.7×10^{-3}
NRSur7dq4, against LALSimulation 6.2.0	200/200	2.8×10^{-12}	3.0×10^{-13}	3.3×10^{-16}	0
NRSur7dq4, against waveformrs	200/200	2.8×10^{-12}	3.0×10^{-13}	3.3×10^{-16}	0

Random binaries drawn as in waveformrs's report (section 10). IMRPhenomXPNR is evaluated on 4,096 uniformly spaced frequencies per draw. "Draws" counts the draws compared. Mismatches use the aLIGO design sensitivity without maximisation.

In short:

- **Implementation.** 134 Python modules (34,120 lines, of which 3,911 are parameter-space fits generated from waveformrs's Rust by a translator) cover the six models, in JAX and MLX, their checks, the conversions and the matches (sections 2–8).
- **Accuracy.** NRSur7dq4 agrees with LAL at rounding level. IMRPhenomXPNR agrees with LAL with the parity fix to a median mismatch of 2.8×10^{-8} where LAL keeps the SpinTaylor angles; it differs by more where LAL switches to other angles, which the model does not use, and by up to ~2×10^{-4} for some heavy binaries, where it attaches the merger-ringdown angles with exact derivatives rather than LAL's spline ends (section 14). The other models agree with LAL to rounding or close to it (section 10).
- **Single precision.** Every model runs in float32 when the binary's parameters are float32, and returns the strain times 10^{21} by default. For NRSur7dq4 in float32, an inner product between two different waveforms changes by 2.4×10^{-6} of the product of their norms (median; at most 1.0×10^{-4}). Derivatives agree with float64 to 5.6×10^{-5} (section 7).
- **Apple GPUs.** NRSur7dq4 also runs in MLX on Apple GPUs, in float32, with its derivatives, LAL's grids and the conditioned waveforms. It is as accurate as JAX's float32 version: the overlap of two binaries changes by 2.3×10^{-6} (median). In batches it evaluates 41,208 waveforms per second, 27.7× JAX's best on this machine's CPU (section 7).

- **IMRPhenomXPNR on Apple GPUs.** IMRPhenomXPNR runs in MLX, in float32, with its derivatives, generated from the JAX model's own jaxprs. Its mismatch with float64 is as small as JAX's float32 one (6.4×10^{-9} median). In batches it evaluates 3,626 waveforms per second, 26.0× JAX's best on this machine's CPU (section 3).
- **IMRPhenomXAS.** The aligned-spin (2,2) model is public, written for speed in JAX and in MLX: each binary's coefficients form a table, and each frequency costs a short formula. It agrees with LAL to 2.4×10^{-10} in float64. In float32 the leading term of the inspiral phase is computed in extended precision, so binary neutron stars from 20 Hz keep mismatches of at most 7.4×10^{-8} in MLX and 3.4×10^{-8} in JAX. One binary on 2^{20} frequencies takes 5.75 ms in JAX, 8.6× faster than LAL, and 566 µs in MLX (section 4).
- **IMRPhenomXHM.** The aligned-spin model with higher modes is public in the same form, LAL's current release on one code path, with its modes as well as its polarizations. Its polarizations agree with LAL to mismatches below 5.1×10^{-8} in float64; in float32 binary neutron stars from 20 Hz keep mismatches of at most 7.3×10^{-8} in MLX. In MLX batches of 4096 binaries on 1000 frequencies reach 543,075 waveforms per second (section 5).
- **TaylorF2 and IMRPhenomD.** Both are public in the same form as IMRPhenomXAS, in JAX and in MLX. TaylorF2 agrees with LAL to 3.8×10^{-10} in float64. IMRPhenomD replaces LAL's interpolated table of quasi-normal-mode frequencies with closed-form fits in the final spin, no less accurate than LAL's spline, and agrees with LAL to mismatches below 6.6×10^{-10}. In MLX, batches reach 5,915,656 (TaylorF2) and 4,915,199 (IMRPhenomD) waveforms of 1000 frequencies per second (section 6).
- **Against NRSur7dq4.** `waveformx.match` gives matches maximised over time, phase or polarisation as pure JAX functions, and optimises them over the conventions that differ between models. On a fixed set of 40 binaries at 90–200 $M_\odot$, our IMRPhenomXPNR departs from LAL 6.2.0's almost only through the parity fix. Where LAL's IMRPhenomXPNR and NRSur7dq4 disagree, ours is closer to NRSur7dq4 for 14 of 20 binaries (median mismatch ratio 0.89); where they agree, for 9 of 20 (section 11).
- **Smooth in the parameters.** IMRPhenomXPNR is continuous where LAL switches discretely: through vanishing in-plane spins (it changes by 10^{-12} for an in-plane spin of 10^{-12}), at equal masses and across LAL's thresholds in the angles. The steps that belong to LAL's model rather than its numerics, such as that of the (3,3) mode across zero total aligned spin, are kept (section 3).
- **A reference scaled by the mass.** With `reference_mf` every model gives the binary at the frequency `reference_mf` /M, the same point of the inspiral for every mass; converters map binaries between this convention and LAL's. For IMRPhenomXPNR the waveform is then independent of the reference chosen (section 3).
- **Derivatives.** IMRPhenomXPNR is differentiated in forward mode, and its reverse-mode gradient is the transpose of that forward-mode Jacobian. NRSur7dq4 is differentiated directly in either mode. NRSur7dq4's derivatives agree with finite differences to ~1e-8, IMRPhenomXPNR's to 1.9×10^{-7} at worst (section 12).
- **Cost.** A compiled IMRPhenomXPNR waveform takes 7.81 ms, against 13.82 ms for LALSimulation. `jax.vmap` over many binaries is how JAX code would use it. Compilation takes 11.9 s for the model, 17.9 s under `jax.vmap` and 47.4 s for its gradient. Long waveforms and batches are evaluated in chunks and blocks, so the memory of a `jax.vmap` of Jacobians stays within a few GB (section 13).

2 Models as pure JAX functions

The package has the interface of waveformrs's Python bindings, with the same function names, keyword arguments, options, result classes and exceptions. Underneath, every model is a pure function of a `Binary` : a `NamedTuple` of float64 arrays (or float32 arrays), and so a JAX pytree.

```
import jax, jax.numpy as jnp, numpy as np
import waveformx as wf

frequencies = np.arange(20.0, 1024.0, 0.125)
hp, hc = wf.polarizations("IMRPhenomXPNR", frequencies, mass_1=36.0, mass_2=29.0,
                          spin_1=(0.4, 0.2, 0.3), spin_2=(-0.2, 0.3, -0.1), f_ref=20.0)

# the pure function, for jit, vmap and grad
model = wf.polarizations_function("IMRPhenomXPNR")          # f(binary, frequencies)
binary = wf.Binary(mass_1=36.0, mass_2=29.0, spin_1=jnp.array([0.4, 0.2, 0.3]),
                   spin_2=jnp.array([-0.2, 0.3, -0.1]), f_ref=20.0)
loss = lambda b: jnp.sum(jnp.abs(model(b, frequencies)[0]) ** 2)
jax.grad(loss)(binary)                                      # a Binary of derivatives
jax.vmap(model, in_axes=(0, None))(binaries, frequencies)    # many binaries at once

# NRSur7dq4 gives the strain times 1e21 (scaled_strain=False: the strain itself)
h = wf.td_polarizations("NRSur7dq4", 1 / 4096, 20.0, mass_1=40.0, mass_2=20.0)
surrogate = wf.td_polarizations_function("NRSur7dq4")       # f(binary, times)
single = wf.Binary(mass_1=jnp.float32(40.0), mass_2=jnp.float32(20.0), spin_1=jnp.zeros(3, jnp.float32),
                   spin_2=jnp.zeros(3, jnp.float32), f_ref=20.0)
hp, hc = surrogate(single, h.times)                         # in float32
h = wf.simulate_td("IMRPhenomXPNR", 1 / 4096, 20.0, mass_1=36.0, mass_2=29.0)
```

A call goes through four stages:

1. **Checks on the host**
   The public functions convert the arguments and run LALSimulation's checks on concrete values (`MODEL.validate`), outside any JAX transformation. Invalid masses, spins or frequencies raise the exceptions of waveformrs: `InvalidParameterError` , `ModelError` and so on.
2. **A compiled function per configuration**
   The model method is compiled with `jax.jit` with the options fixed. `WaveformOptions` is a frozen dataclass, so it can be a static argument. JAX caches the compiled function per options and array shape, so a sampler that keeps its frequency grid compiles once.
3. **Traced arithmetic**
   Inside, nothing branches in Python on the binary. Each of LAL's `if` s becomes a `jnp.where` between branches that are both evaluated, with safe arguments, so that the branch not taken cannot put NaN into the value or the gradient. Loops of fixed length are `lax.scan` ; loops whose length depends on the binary are `lax.while_loop` .
4. **Failures as NaN**
   Failures that can only be detected while computing make the output NaN. The batch functions report those binaries as `ModelError` in `errors` and set their rows to zero.

Importing the package enables 64-bit floats (`jax_enable_x64`), which IMRPhenomXPNR's comparisons with LAL need; both models also run in float32 (section 3, section 7). The thousands of fitted coefficients of IMRPhenomX are not copied by hand. `scripts/rust_to_python.py` parses the straight-line Rust of waveformrs (its `let` bindings, arithmetic, `fma` calls and `f64` methods) and emits the same expressions on JAX arrays. `scripts/generate_fits.py` then writes them into `models/xpnr/_generated/` . The control flow is written by hand, file by file after LAL's, and each module's docstring names the LAL function it follows.

Each model package defines a `MODEL` with a fixed contract. A frequency-domain model has `validate` , `polarizations` , `modes` and `choose_fd_modes` . A time-domain model has `validate` , `td_polarizations` , `td_modes` and `polarizations_at` , the last of which evaluates the waveform at any given times. The public functions, the batches and the conversions are written once, against that contract.

3 IMRPhenomXPNR

IMRPhenomXPNR is IMRPhenomXPHM with the phenomenological PNR precession angles, a co-precessing (2,2) mode tuned to numerical relativity, an antisymmetric (2,|2|) contribution and the 122019 release of the higher modes. In LALSimulation it draws on many of the 49 IMRPhenomX source files, together with the SpinTaylor code and GSL.

LAL's physics, not its numerics

Up to commit `0bbd3c5` (tag `lal-faithful`) the package reproduced LALSimulation's numerical procedure step for step: GSL's adaptive RKF45 integration of the SpinTaylor equations replayed one step at a time, LAL's splines through its samples, the fallbacks to the NNLO and MSA angles, the released model without the parity fix, the roots of GSL's companion matrix, LAL's finite differences in the higher modes, single-precision storage and masses in kilograms. That version agrees with LAL 6.2.0 to the level at which different builds of LAL agree with each other, and its report describes how (below). Since then the model keeps LAL's physics, the fits, the ansatzes, the PNR construction of the angles and the calibrated choices of the higher-mode amplitudes, and computes it in the way best suited to JAX: the comparisons with LAL go through LAL itself or waveformrs, which reproduces LAL to rounding. What differs from LAL:

- **One prescription.** LALSimulation chooses the Euler angles by the binary: in-plane spins below 1e-7 give plain IMRPhenomXPHM with the NNLO angles, below 1e-6 the PNR machinery runs with the NNLO angles (version 102), and where its SpinTaylor integration fails it falls back to the MSA angles (223), or to NNLO if the MSA system cannot be set up. The model uses the SpinTaylor angles (330) with the PNR machinery for every binary, so it compiles one prescription and is continuous through vanishing in-plane spins. Versions 102 and 223 are models of their own, `IMRPhenomXPNR_NNLO` and `IMRPhenomXPNR_MSA` , each used for every binary (below).
- **The SpinTaylor solution.** The precession equations are solved once, in one Dormand–Prince 8(5,3) loop with dense output (`spintaylor/lean.py`): backward steps of equal increments of $u = 1/(M\omega)$, so that each step covers about the same precession angle, to below f_{min}, then forward steps from f_{ref} (at most a tenth of u each, which keeps the steps short near $v = 1$ for large aligned spins) until LAL's stopping conditions are met. A few hundred steps resolve a whole inspiral to ~1e-12, and the step size is a smooth function of the state, so the solution is differentiable. The angles are evaluated from the dense output, with no sampling lattice and no splines through millions of samples. The integration starts low enough for the grid of the PNR interpolants, so LAL's second integration from the start of that grid is not needed. The end of the inspiral is where the first stopping condition is met on the dense output, less half a step of LAL's fine sampling, found by bisection and refined by a Newton step that carries the derivatives. Where the stopping conditions already fail at the reference frequency (beyond the validity of the PN equations, where LAL gives up the SpinTaylor angles), the end is where they are first met on the backward solution.
- **Continuous switches.** LAL's discrete switches in the angles are made continuous: the hand-off of the SpinTaylor α to its merger-ringdown continuation (at 0.97 of the end of the inspiral, where the two agree in value and slope, rather than 0.98), the two-spin construction (for every binary, rather than from $|\chi_2| =$ 1e-3), the remapped J frame (for all in-plane spins, rather than from 1e-3), the refinement of the grid of the PNR interpolants near $L \approx J$, the edge of the PNR transition region at $q = 12$ (blended over 12–14) and the choices of the connection frequencies of the PNR β (blended across bands of 0.002 in Mf). The PNR angles are interpolated with LAL's splines of α, β and γ: splines of the unit vector of L and of $\gamma \pm \alpha$ would follow L passing close to J, but where LAL's grid lets α turn by up to ~3 rad between nodes, at its lower end, they cut across its circle (mismatches with LAL of up to 5×10^{-4}), and refining the grid until they do

not costs more than LAL's splines. They remain for the phase grid. LAL's splines are taken on a grid of its own rather than on LAL's uniform grid, whose spacing is what its lower end needs (0.01 Hz, its floor, for light binaries: up to ~300,000 nodes): the nodes span LAL's band with a spacing that grows like f^2 (constant steps of the precession and nutation phases at leading order) from a quarter of LAL's spacing up to 10^{-4} in Mf, and the splines are taken in the index of the nodes. A few thousand nodes then follow LAL's splines to mismatches of 10^{-10} at most (median 10^{-16} over random binaries), closer than LAL's own grid is to its limit where that grid is coarse (3×10^{-8} for a 12.6 + 12.1 $M_\odot$ binary). The quarter of LAL's spacing at the start covers L passing close to J above the start, where LAL's grid is finer than it needs to be at the start; with LAL's spacing the f^2 grid differs from LAL's by up to 3×10^{-6} for light binaries of $q \approx 10$. The antisymmetric mode reads the PNR α below its transition from the same splines, plus the constant by which LAL's second evaluation of α there differs (the change of the offset of α), rather than evaluating the SpinTaylor α again at every frequency. A separate report (`report/extended` , measured at the tag) maps these switches and their sizes.

- **Vanishing spins.** As the in-plane spins vanish, the waveform tends linearly to that of the aligned spins, and its derivative along each direction tends to the one-sided derivative there. Until this change it jumped by ~1e-3 between an in-plane spin of 1e-8 and none (at $\iota = 0.7$; 2e-6 face-on), because LAL's construction leaves an opening angle where the in-plane spins vanish, about an α that is arbitrary there. LAL's arctan window (Eq. 62 of arXiv:2107.08876) keeps the PNR β at least 1.4e-3 rad from zero. The single-spin MSA system behind the taper of the two-spin oscillations perturbs its degenerate roots by 1e-9, which leaves $\beta \approx 3e\text{-}5$. And the polar angle of the effective spin, kept 1e-15 from the poles, leaves an in-plane spin of ~5e-8 in the PNR fits. The model keeps β as it is within $[0, \pi]$ (its splines of the unit vector of L are smooth through the poles) and goes over into LAL's window across 0.01 rad outside, where the window folds the merger-ringdown fits back. It evaluates the single-spin β as $\mathrm{atan2}(S_\perp, L + S_\parallel)$, LAL's MSA expression for a spin of constant magnitude without the perturbed roots, and gives the PNR fits of the form $\chi \sin\theta\, g(\cos\theta)$ and the PN waveform β the in-plane spin itself. The SpinTaylor β, the J frame, θ_{JN} and the polar angles of the spins are taken as atan2 of components rather than as acos of cosines, which lose half the digits of small angles and with them the derivatives. Now the waveform changes by 1e-12 for an in-plane spin of 1e-12, and as both spins vanish together it changes linearly too, except across zero total aligned spin, where the (3,3) mode switches (below). At zero in-plane spin itself the waveform has no derivative: LAL's fits of the co-precessing (2,2) mode and of the PNR angles depend on the magnitude of the in-plane spin, so the one-sided derivatives along opposite directions differ. There the derivatives are finite. At equal masses LAL's estimate of the period of the two-spin oscillations is NaN, and its grid of the PNR interpolants the coarse one. The limit $q \to 1$ is the finest grid, which the model takes, removing a step of ~1e-3 at $q = 1$ (`tests/test_xpnr_smooth.py`).
- **Numerics.** Masses stay in solar masses, the mixing coefficients of the (3,2) mode are not rounded to single precision, nothing keeps XLA from fusing multiply-adds, the connection frequencies of the PNR β are evaluated with finite stand-ins for LAL's NaNs (without replaying its NaN arithmetic), the higher modes take exact derivatives where LAL takes finite differences with steps of 1e-9 and 1e-7, and the quintic of the higher-mode amplitudes is tested for a zero crossing by its signs at 257 points instead of the roots of GSL's companion matrix.
- **The parity fix.** The model includes the fix of lalsuite !2593 from LALSuite's development branch, as waveformrs does by default. It restores the mirror symmetry of the antisymmetric modes: flipping the in-plane spins and taking $\iota \to \pi - \iota$ gives $(h_+, -h_\times)$. The released LALSimulation 6.2.0 breaks it by more than 1e-4.

The physics switches that LAL makes on the binary in the higher modes are kept: the versions of the inspiral and intermediate amplitudes chosen by mass ratio and spin, the vetoes of collocation points, the lowering of the order of the intermediate polynomial where it would cross zero, the second intermediate region for extreme mass ratios (η thresholds), the inspiral ansatz in the ringdown of the (3,3) and (4,4) modes, the collocation points of the intermediate phase of the (3,3) mode, which change with the sign of the total aligned spin (a step of ~1e-3 in that mode across zero total spin), and the vanishing odd modes of equal masses with equal spins, which the odd modes approach continuously. They are LAL's model rather than its numerics; making them continuous would change the physics.

Against LAL with the parity fix the model agrees to mismatches of 1e-11 to 5e-5 where LAL uses the SpinTaylor angles (closest for two generic spins away from LAL's switches; up to ~2e-4 beyond $q = 12$, where the angles go smoothly to LAL's outside the transition region), and differs by up to a mismatch of ~0.3 where LAL falls back to the MSA or NNLO angles (section 10). Measured one at a time (`report/xpnr_cost.py` : 64 frequencies, all modes, tracing + lowering + compilation), it compiles in 11.9 s, under `jax.vmap` in 17.9 s and its gradient in 47.4 s, against 14.4 s, 19.6 s and 56.7 s for the version at the tag; at the benchmark binary it runs in 6.24 ms (10.9 ms at the tag).

The option `extended_phase_grid` interpolates the PNR angles with splines of the unit vector of L and of $\gamma \pm \alpha$ on such a grid from LAL's spacing (rather than a quarter of it). For the benchmark binary it runs in 8.55 ms (against 6.24 ms); for a light binary (8 + 5 $M_\odot$ from 10 Hz on 16,225 frequencies) in 22.09 ms against 19.78 ms, at mismatches below 1e-4 from the default (`tests/test_xpnr_smooth.py`). It compiles in 12.2 s (47.9 s for the gradient).

The NNLO and MSA prescriptions

LAL's other two precession prescriptions are approximants of their own: `IMRPhenomXPNR_NNLO` builds the PNR angles on the NNLO angles (`PhenomXPrecVersion` 102) and `IMRPhenomXPNR_MSA` on the MSA angles (223). They are names rather than an option because the option LAL's dictionary offers, `precession_version` , already has waveformrs's meaning for IMRPhenomXPHM (default 320, and 223 in `lalsimulation_defaults()`); an approximant is static, with a compiled function of its own, and every public function, the conversions, the converters between reference conventions and the MLX port take it. The prescription is a static `version` of `model.polarizations` . Neither integrates the SpinTaylor equations: the set-up is `finish_setup` , `populate_structs` and the grid of the PNR interpolants of that version, and the rest of the model is shared. Each is used for every binary, without LAL's switches between versions (below in-plane spins of 1e-6 LAL's 223 takes the NNLO angles, and below 1e-7 both turn the PNR angles off); where the MSA system cannot be set up (LAL then takes the NNLO angles) the MSA model is NaN and the public functions raise `ModelError` . The converters between reference conventions are IMRPhenomXPNR's, which evolve the spins with the SpinTaylor equations.

Against LAL 6.2.0 with the parity fix at versions 102 and 223, on waveformrs's suite of 32 binaries (`tests/test_xpnr_prescriptions.py` ; waveformrs itself keeps version 330 for IMRPhenomXPNR, so the reference is a patched LAL, `scripts/xpnr_prescriptions_reference.py`), the mismatches are 10^{-16}–10^{-10} for most binaries, closer than the SpinTaylor model, whose solution of the precession equations differs from LAL's. The exceptions are the choices the model makes continuous: in-plane spins below 10^{-6} (up to 0.09, where LAL's 223 with aligned spins is not even the limit of its own waveforms), the edge of the PNR

transition region at q = 12 (up to 1.6×10^{-3}), the window of the PNR β at small in-plane spins (~4×10^{-7}), the two-spin construction for every binary (3×10^{-8}), and one binary (5×10^{-4} for NNLO) where the extrema of the merger-ringdown β are complex: LAL's arithmetic gives NaN there and drops the connection of β, which moves the frequency map of the higher modes, while the model takes the finite limit, as the SpinTaylor model does. And for three heavy binaries (250–480 $M_\odot$) the NNLO model differs from LAL by mismatches of 2×10^{-6}–3.5×10^{-5} because LAL's uniform grid for the NNLO angles is too coarse for its splines: made 4 or 16 times finer it moves by the same amounts, to within 5×10^{-8} of the model's grid.

The MSA β as the in-plane spins vanish. LAL's two-spin MSA β is $\cos^{-1}$ of $(J^2 + L^2 - S^2)/(2JL)$, with $|S|^2$ oscillating between two roots of a cubic. It does not vanish with the in-plane spins, for two reasons. Where the two roots nearly coincide, the rounding of the coefficients of the cubic moves them by ~$\sqrt{\varepsilon}$ of its scale, and β by up to ~10^{-4} rad, a step that does not shrink with the in-plane spins. LAL perturbs exactly degenerate roots (a spin along L) by 10^{-9}, which leaves $\beta \approx 3\times10^{-5}$. And for binaries whose heavier spin points along L and lighter one against it, the third root rises through the other two late in the inspiral (the up-down instability): the MSA lets the oscillation span a range of $|S|^2$ that does not shrink with the in-plane spins, as if the instability grew at once from any misalignment, and β tends to ~0.1 rad. The model takes the β of a total spin of constant magnitude, atan2($|S_\perp|$, $L + S_\parallel$), LAL's construction for degenerate roots without the perturbation, where the separation of the roots is small, and blends LAL's in across a decade of a well-conditioned estimate of it ($4|S_{1\perp}||S_{2\perp}|$ relative to the scale of the cubic, vanishing as ~12δ at equal masses), in-plane spins of ~3×10^{-3} to 10^{-2} (`pnr._msa_beta`). Above that the model is LAL's. Below it, it departs from LAL's procedure by up to a mismatch of ~10^{-5} (seen nearly edge-on) by the oscillations of |S| it drops, and by up to ~2×10^{-3} for up-down binaries, where LAL's own waveform is discontinuous at zero in-plane spin. Both prescriptions then tend linearly to the waveform of aligned spins as the in-plane spins vanish, as all spins vanish and across equal masses, and are continuous across LAL's switches at $\chi_\perp = 10^{-3}$ and q = 12 (`tests/test_xpnr_smooth.py`). The common set-up now also takes the in-plane spins without the floor of ~10^{-154} of `hypot2`, whose square versions 102 and 223 divide by, which gave NaN derivatives for aligned spins. Their gradients agree with finite differences as the SpinTaylor model's do (`tests/test_transforms_xpnr.py`), and the MLX port runs them from the same generated stages, with one stage in place of the SpinTaylor solution and the set-up of version 330.

Cost. Measured as `report/xpnr_cost.py` measures the SpinTaylor model (64 frequencies, all modes, tracing + lowering + compilation in a fresh process without the compilation cache), on 2026-10-25 with other work on the machine (load ~4.6): the NNLO model compiles in 9.4 s, under `jax.vmap` in 13.4 s and its gradient in 38.3 s, the MSA model in 11.6 s, 16.8 s and 51.9 s, against 13.0 s, 19.1 s and 50.0 s for the SpinTaylor model in the same run. The MSA system, its roots and elliptic functions carry their own derivatives, so its gradient compiles no faster than the SpinTaylor model's. At the benchmark binary (8033 frequencies) they run in 3.2 ms and 5.4 ms against 7.5 ms. These are not in `report/data`.

A light binary. For 5 + 1.5 solar masses from 10 Hz with 1/128 Hz resolution (260,865 frequencies, all modes, the same machine and load), the NNLO model runs in 53 ms, of which its angles take ~4 ms: its grid of the PNR interpolants, at LAL's spacing of 0.45 Hz for the NNLO angles, has 5,380 points; the rest is shared with the other models. The MSA model's grid follows the two-spin oscillations at LAL's floor of 0.01 Hz, 244,000 points, at each of which it evaluates the MSA angles, and its antisymmetric (2,2) mode evaluates the MSA α at every frequency below its transition. Two changes, neither of which changes the waveform beyond rounding (3×10^{-14}), take it from 248 ms to ~220 ms: the MSA angles are evaluated once per point of the grid, for α, and handed to the variants of β (each computed under its own branch, which recomputed them), and the Jacobi elliptic function sn of the oscillations skips those of GSL's 15 AGM steps that no point needs (about 11), with the same arithmetic in the steps it takes. What remained, ~45 ms in the grid and ~45 ms in the antisymmetric mode, was the arithmetic of the MSA expressions, mostly their transcendental functions; the grid of nodes of the PNR interpolants (a few thousand rather than 244,000, above) and the antisymmetric α from its splines remove both: the MSA model now runs in 43 ms and the NNLO model in 41 ms (55 ms with LAL's uniform grid), on an idle machine. The phase grid (`extended_phase_grid`) takes the MSA model to ~117 ms at a mismatch of 1.7×10^{-7} from the uniform grid. For the NNLO angles it started at LAL's coarse spacing, across which the NNLO α advances by tens of radians at low frequencies, more than the interpolants of the unit vector of L can follow (a mismatch of 0.38 for this binary, without an error flag); it now starts at a spacing over which α advances by at most 0.25 rad (mismatches of 10^{-10}–10^{-9} from the uniform grid), and costs more than the uniform grid for these angles.

Branches and loops under `jax.vmap`

Where LAL's code branches on the binary, both branches are evaluated with safe arguments and selected, or, for expensive branches, computed under the model's own `switch`, `cond` and `while_loop` (`_transforms.py`). Under `jax.vmap`, `lax.cond` with a batched predicate would compute every branch for every binary; these have batching rules instead: a branch is computed for the whole batch only if some binary in the batch takes it, and a loop runs to the longest binary in the batch while selecting only its small carry at each iteration. The three variants of the PNR β (in the calibration region with the merger-ringdown β attached, in the transition region, and without it) are such branches.

The antisymmetric amplitude

LAL smooths the amplitude ratio of the antisymmetric mode with an 81-point moving average computed in place, so each window reads averages it has already written. That is an order-40 linear recurrence, part of the definition of the waveform, which the model keeps. It is evaluated in blocks of 40 values with a triangular solve per block, inside a `lax.scan`; on a uniform frequency grid one system serves all blocks.

Derivatives in forward mode

Loops whose number of iterations is traced, such as the SpinTaylor integration, can be differentiated in forward mode but not in reverse mode. Reverse mode through the model's many scalar set-up computations also compiles slowly. So `_transforms.differentiable` makes the model a `jax.custom_jvp` function. Its rule computes the Jacobian with respect to the parameters that have non-zero tangents, one `jax.jvp` per parameter, and contracts it with the tangents. The contraction is linear, so `jax.grad`, which transposes it, works as well as `jax.jvp` and `jax.jacfwd`. The Jacobian is itself a `custom_vmap` function, so under `jax.vmap` of a gradient the binaries are batched inside the differentiation, four at a time (sixteen for the waveform itself: memory).

History: reproducing LAL's numerics

The version at the tag `lal-faithful` (commit `0bbd3c5`) reproduced LAL's numerical procedure, with the SpinTaylor angles alone by default and LAL's fallbacks (`version_fallback`), the released model without the parity fix (`pnr_parity_fix`), the extended angles (`extended_spintaylor`) and the present model (`lean`) as options. Its forward and fine SpinTaylor runs repeated GSL's adaptive RKF45 steps one for one in a `while_loop`, with GSL's step control, LAL's retries and its Hermite-like interpolant, so that each run ended where LAL's ends, with the same samples; the backward run was the dense Dormand–Prince solution; and its splines had as many knots as LAL has samples, evaluated from the 34 knots around each point beyond 8192 knots. It reproduced GSL's Jacobi elliptic functions and elliptic integrals for the MSA angles, GSL's balanced companion matrix and Francis QR iteration for `CrossZeroP5` (testing every root with its own parts, where LAL reads shifted entries), LAL's NaN arithmetic in the connection of the PNR β (a first pass with LAL's arithmetic under `stop_gradient` to take the decisions, a second with finite stand-ins following them), and LAL's finite differences in the higher modes, whose tangents were those of the exact derivatives. Its four prescriptions were branches of the model's `switch`, versions 102 and 223 of the PNR machinery sharing one branch. It agreed with LAL 6.2.0 to ~1e-5 (relative error over 200 random binaries, mismatch below ~1e-9): the (2,2) mode to ~1e-9 and the higher modes to the rounding of 1e-16 amplified by LAL's finite differences. Making it compile and run faster took three rounds of work: forward-mode derivatives, the batch-aware branches and loops, one evaluation of the SpinTaylor splines for all the points the set-up reads, the coefficients of the four higher modes under one `jax.vmap`, optimization barriers against XLA's duplication of shared computations, and loop bodies kept within the limits under which XLA's CPU runtime runs them on the calling thread (`_math.isolated_step`). The model kept every one of these.

A reference frequency scaled by the mass

LAL gives the spins, the inclination and $\phi_{\rm ref}$ at a reference frequency $f_{\rm ref}$ in Hz, by default the first frequency, and requires $f_{\rm min} \le f_{\rm ref} \le f_{\rm max}$ for the PNR angles. The SpinTaylor angles are needed over the band of (2,2)-equivalent frequencies $2f/m'$ of the modes, and LAL starts that band at $2f_{\rm min}/M_{\rm MAX}$, with $M_{\rm MAX}$ the largest m' requested. For the (2,1) mode alone the band starts near $2f_{\rm min}$, above a reference at $f_{\rm min}$: LAL leaves the SpinTaylor angles (for the MSA angles) and the model gives NaN. Of 256 random binaries ($q \le 30$, 3–400 solar masses, spins ≤ 0.99, bands from 10–30 Hz), the model with $f_{\rm ref} = f_{\rm min}$ is finite for 4 with the (2,1) mode alone and 256 with all modes.

The option `reference_mf` gives the binary at $f_{\rm ref} =$ `reference_mf` $/M$ instead, the same point of the inspiral for every binary; `Binary.f_ref` must then be zero (the public functions raise an error otherwise, the pure functions give NaN, and the derivative with respect to it is zero). The recommended value, `REFERENCE_MF` $= 0.01$, lies before the end of the inspiral for every binary: with the spins given there, the PN energy test ends the inspiral at $Mf \ge 0.015$ for $q \le 30$ and spins ≤ 0.99 (the worst case: $q = 30$, both spins -0.99 along L), so the reference is at most 2/3 of the way there, below the hand-off of the SpinTaylor α to its merger-ringdown continuation at 0.97 of it. What changes (`pnr.common_setup`, `pnr.interpolation_grid`):

- **The band of the angles reaches the reference.** All quantities are (2,2)-equivalent frequencies, and the minimum is taken after the scaling by $2/m'$. The grid of the PNR interpolants holds the bands of the requested modes only (LAL's always holds the (2,2) band, below the integration where $m' = 1$ alone), extended by soft minima and maxima (C^2 blends a node wide) to hold $f_{\rm ref}$ with at least four nodes on each side; for heavy binaries, whose spacing is a sizeable fraction of the frequency, the spacing is at most $f_{\rm ref}/8$. The integration starts below min(LAL's start, $0.25\, f_{\rm ref}$), again by a soft minimum, and at $0.25\, f_{\rm ref}$ where LAL's start is not positive (heavy binaries with $m' = 1$ or 3 alone).
- **Everything else follows from $f_{\rm ref}$.** The phase alignment of the (2,2) mode, the line of sight, the J frame, the offsets of α and ε and the PNR splines at the reference read the waveform struct's $f_{\rm ref}$, so they move with it. The model in LAL's convention is unchanged (static branches).

With the mass-scaled reference the model is finite for at least 256 of the 256 binaries for every mode array ((2,1), (3,2), (3,3), (4,4) alone, and all modes), and its gradients are finite in float32 and float64. Along the total mass across the masses where $f_{\rm ref}$ crosses the lower end of the band (~200 solar masses for all modes at $f_{\rm min} = 20$ Hz, ~50 for the (2,1) mode alone) the overlap shows no jump (fifth differences within 8 times their noise) and the forward-mode derivative agrees with finite differences to 2×10^{-3}. In float32 the waveform agrees with float64 to mismatches below 3×10^{-6}, and the gradients are as close to float64 as with LAL's convention. Compilation is unchanged (12.7 s against 12.9 s, cache off), and the run time is within ~10% up to 150 solar masses (at 10, 60 and 150: 21.16 ms, 6.52 ms, 6.18 ms against 21.10 ms, 6.74 ms, 5.74 ms); heavier binaries, whose reference lies below the band, integrate further down and interpolate on a finer grid (6.56 ms against 5.05 ms at 400 solar masses). The MLX port takes the option too and agrees with JAX float64 as closely as JAX float32 does.

Converters. `to_mass_scaled_reference` and `to_physical_reference` (`models/xpnr/reference.py`) map a binary between the conventions by integrating the model's SpinTaylor equations between the two references (4096 DOP853 steps in $u = 1/(M\omega)$), as pure functions (jit, vmap, grad). They keep the spins, L and the line of sight. The frame at the new reference is the one the equations carry along, turned about L by the orbital phase between the references, taken from IMRPhenomXAS's (2,2) phase, by which the model's alignment of the phase moves. With that turn, aligned-spin binaries give the same waveform in both conventions (mismatch ~10^{-11} without maximisation), and round trips return the binary to ~10^{-14}. The orbital phase is thousands of radians for light binaries, so in float32 the converted spins and angles are good to ~10^{-4}: convert in float64. The basis of the polarizations follows L at the reference, so the two descriptions differ by a rotation of the polarizations, which the converters return.

For precessing binaries the conventions differ by more than numerics, because IMRPhenomXPNR itself depends on where the reference lies: it takes the aligned spins of the co-precessing modes, J and the parameters of its fits at the reference. Over – binaries drawn at 20 Hz ($q \le 12$, 20–90 solar masses, spins ≤ 0.9) and converted to $0.01/M$ (inside the band), LAL with the parity fix at the two references differs by a mismatch of median – and maximum – (above 10^{-3} for –), maximised over time and phase. The model with the mass-scaled reference agrees with LAL given the converted binary at $f_{\rm ref} = 0.01/M$ to a median of – and a maximum of –, as closely as the model agrees with LAL in its own convention. As the in-plane spins vanish, the waveforms of the two conventions converge (mismatch 4×10^{-5} at $\chi_{\rm p} = 0.3$, 10^{-8} at 0.01). A converted binary therefore stands for the same physical system, but a waveform model whose reference moves is a different waveform for precessing binaries, by as much as LAL's own dependence on $f_{\rm ref}$.

A reference-invariant model. The model with the mass-scaled reference removes that dependence: whatever its `reference_mf`, it first carries the binary to the canonical reference `REFERENCE_MF` $= 0.01$ (`reference.to_canonical`, the converter's integration with a step count fixed by the two references, so the MLX port

takes it too), and the rest of the model runs as at 0.01. The aligned spins of the co-precessing modes, J, the parameters of the PNR fits, the offsets of the angles, the alignment of the phase, the line of sight and the basis of the polarizations are then all taken at 0.01. Reading the state at 0.01 from the model's own SpinTaylor solution instead would be circular, since the start and end of that integration and the grid of the PNR interpolants follow from the aligned spins at the reference. The converters pass through 0.01 too, with the orbital phase of both legs from the aligned spins there, so conversions compose. A binary converted between two references (0.006 and 0.012, 0.01 and 0.008, 0.01 and 0.002; precessing, nearly aligned and aligned) gives the same waveform without any maximisation, to a largest relative difference of 10^{-13} (10^{-7} at q = 18, whose integration takes few steps), in float64, float32 and MLX; without the conversion the waveform changes by order one. Before this, the model at two references differed by mismatches of 10^{-5}–2×10^{-2}, as LAL does. Two checks show that this is the model's construction and not the converter. The converter's evolved spins, L and E1 agree with the model's own SpinTaylor solution to 10^{-13} (10^{-8} for q = 18), and with LAL's `SimInspiralSpinTaylorPNEvolveOrbit` to 10^{-9}. With `reference_mf` = 0.01 nothing changes. Another value costs an integration between the two references and a compilation of its own (the option is static): 14.3 s and 14.4 s to compile at 0.006 and 0.002 against 12.7 s, and at 10, 60 and 400 solar masses 21.73 ms, 6.69 ms, 7.03 ms at 0.006 and 23.19 ms, 8.44 ms, 8.80 ms at 0.002. For a reference given in Hz, `to_mass_scaled_reference` followed by the model at 0.01 gives the same waveform without recompiling.

Against LAL. The invariant model is compared with LAL where LAL's reference is the canonical one, f_{ref} = 0.01/M: there both take everything at the same point, and they agree to a median of – and a maximum of – (above). Elsewhere the difference is LAL's own dependence on its reference. With the same spins and angles given to LAL at $f_{ref} = M_{near}/M$, off 0.01 by a relative offset drawn log-uniformly in [0.2%, 20%], the mismatch with the invariant model has median – and maximum – for offsets up to 2% (– binaries), – and – for 2–5% (–), and – and – for 5–20% (–). From LAL's test point at 20 Hz it is median – and maximum – (– of – above 10^{-3}), as LAL at 20 Hz against itself at 0.01/M. LAL's dependence on its reference is not always smooth either: for one binary at q = 11 its mismatch with the invariant model rises from 3×10^{-7} to 5×10^{-5} and falls back to 2×10^{-6} as its reference moves from 24.04 to 24.22 to 24.28 Hz.

Single precision

IMRPhenomXPNR is evaluated in the precision of the binary's parameters, as NRSur7dq4 is, and returns the strain times 10^{21} unless `scaled_strain=False`. The comparisons with LAL use `scaled_strain=False`. A float32 binary is evaluated without promoting any float32 value to float64. Python constants in LAL's expressions remain weakly typed float64 scalars, which take float32 when they meet the data; `tests/test_xpnr_float32.py` checks the waveform, its forward derivative and its gradient. The changes that made float32 work change the float64 waveform by rounding only:

- **No SI round trips.** The masses stay in solar masses in both precisions (at the tag, float64 passed them through kilograms as LAL does; in float32 XLA folds the square of the solar mass in kilograms into the derivatives, as it did for NRSur7dq4). The amplitude takes the distance in Mpc. Two expressions of LAL that XLA rewrites into powers beyond float32's range are written another way: the velocity of the chirp-time bound, where XLA turned a division by c into a multiplication by c^8, and the arctan window of the PNR β, whose 500th powers overflow. The 500th power in that window, below 1.5 for β in [0, 0.01], exceeds float32's range where the ringdown β of the fits is below −0.3 rad (large mass ratios with in-plane spins): its value is finite but its derivative was NaN, so there it is computed as π/2 − arctan of the inverse power, the same function to rounding. The pseudo-PN coefficients of the higher-mode inspiral amplitudes divide by a product of powers of the collocation frequencies, ~10^{-19} for the (2,1) mode at Mf ~ 0.01, whose square in the derivative of the quotient falls below float32's range (NaN derivatives of the (2,1) mode, mostly above q = 10); the coefficients do not depend on the scale of the frequencies, and are computed in units of the end of the inspiral.
- **Constants of the precision.** Machine epsilon, the smallest normal number and the floors of safe square roots (10^{-300} in float64) follow the precision. The higher modes take exact derivatives instead of LAL's finite differences with a step of 10^{-9}.
- **Types from the binary.** Default float and integer types, numpy coefficient tables and the counts of loops follow the precision (`_math.real`, `_math.integer`), since an int64 count or a float64 table promotes float32 data to float64.

The model in float32 agrees with its float64 waveform to a mismatch of ~10^{-9}–3×10^{-7} for the binaries tried, and its gradients agree to ~10^{-4}. Its derivatives are finite in float32 (over 300 random binaries up to q = 30, each mode and the polarizations; before two fixes described above, the derivative of the (2,1) mode was NaN for half of them, mostly above q = 10, and that of every mode for 2%). The version at the tag, which replayed LAL's adaptive RKF45 steps (at tolerances raised to 10^{-5} in float32) and sampled them on LAL's time lattice, differed from float64 by mismatches of 10^{-5}–10^{-2}. Running JAX's float64 switch off inside a trace (`jax.enable_x64(False)`) would have avoided most of the type changes, but JAX caches the conversions of numpy constants and the executables of eager operations without regard to that switch, so a float32 evaluation corrupted later float64 ones in the same process.

On Apple GPUs with MLX

`waveformx.models.xpnr.mlx` runs the model in MLX on the GPU of Apple silicon, in float32: `polarizations(binary, frequencies)` for one binary or a batch (shared or per-binary frequencies), differentiable with `mx.grad`, and `polarizations_and_jacobian`. It is not a second implementation of the model. The model is cut into stages that are functions of one binary's scalars (the set-up: four stages of 7–41 thousand scalar operations) or of its scalars and one point (the PNR angles at a grid point, the angles, modes and twist at a frequency). Each stage is traced to a jaxpr in float32 and lowered (`mlx/_program.py`) to straight-line code on scalars: arrays of static shape become arrays of scalars, loops of static length are unrolled, branches are computed both ways and selected, the small linear solves (LAPACK's `getrf` with partial pivoting) and sorts are written out, complex numbers become pairs, and what does not depend on the inputs is computed once. The code runs as a Metal kernel over the batch and the points, its derivatives from the JVP that JAX writes, one thread per binary, point and direction. Products added once are fused into multiply-adds, as XLA's CPU backend fuses them: without this, the float32 fits of the PNR β and the collocation solves of the higher modes came out 20–100 times further from float64 than JAX's float32. The rest is written for MLX: the SpinTaylor integration (a Metal kernel, one SIMD group per binary with a lane per component of the state, calling the translated derivatives), the searches in its dense output, the natural splines of the PNR interpolants (the inverse of the tridiagonal matrix as a filter of the second differences), Boole's integral of γ and the moving average of the antisymmetric mode.

The derivatives are forward-mode Jacobians with respect to the twelve parameters, contracted with the cotangents in a custom VJP, as the JAX model contracts its own. One choice differs from JAX's float32: the Newton step at the end of the inspiral takes the exact derivative of the stopping conditions in time, where JAX takes the secant of the bisection's bracket, which float32 does not resolve. JAX's float32 derivative of the end of the inspiral is off by up to 90% for the binaries tried; the MLX one agrees with float64 to ~10^{-6}.

Over 200 random binaries of `report/extended` (1024 frequencies each), the weighted mismatch of the MLX waveform against JAX's float64 waveform has median 6.4×10^{-9}, 90th percentile 5.2×10^{-7} and maximum 1.1×10^{-2}; JAX's float32 waveform: 1.1×10^{-8}, 3.1×10^{-7} and 1.1×10^{-2}. 14 binaries exceed 10^{-6} in MLX and 9 in JAX's float32; the largest is the same binary of 411 $M_\odot$ for both (JAX float32: 1.1×10^{-2}). Where they differ, the cause is float32 itself, not the port: LAL's connection of the PNR α takes finite differences of the SpinTaylor α over 0.0005 in M f, which float32 resolves to ~10^{-2}, so its rounding moves the connected α by up to a few hundredths of a radian, more in one evaluation or the other. Both are finite for 200 binaries; 0 are finite in float64 only and 0 in MLX only (1 not in JAX's float32). The gradient of the log-likelihood of `tests/test_xpnr_float32.py` is within that test's bound (10^{-2} |g| + 10^{-3} max |g|) for 27 of 30 binaries, at 5.1×10^{-2} of the bound (median; largest 7.4×10^{0}), against 23 of 30 for JAX's float32 (2.0×10^{-1}, largest 6.0×10^{1}). Data: `report/data/mlx_xpnr.json` (`report/mlx_xpnr.py`).

BINARIES PER CALL	MLX, PER CALL	WAVEFORM, PER BINARY			GRADIENT, PER BINARY		
		MLX	JAX FLOAT32	JAX FLOAT64	MLX	JAX FLOAT32	JAX FLOAT64
1	20.19 ms	20.19 ms	8.07 ms	7.18 ms	113 ms	93.95 ms	104 ms
16	65.47 ms	4.09 ms	14.33 ms	16.76 ms	31.19 ms	126 ms	148 ms
64	90.06 ms	1.41 ms	13.72 ms	17.00 ms	17.94 ms	–	–
256	134 ms	525 µs	–	–	17.25 ms	–	–
1024	282 ms	276 µs	–	–	–	–	–

IMRPhenomXPNR on 8033 frequencies (20–1024 Hz), all modes: MLX on the Apple M2 Ultra GPU, JAX on the CPU under `jax.jit` (one binary) and `jax.jit(jax.vmap(...))` (random binaries of 20–120 $M_\odot$ from 20 Hz). The gradient is that of the power of the polarizations. Median times; the larger JAX batches were not timed.

One waveform takes 20.19 ms in MLX against 7.18 ms in JAX (float64) on the CPU: a single binary is dominated by the sequential integration (one SIMD group runs its steps one after the other) and by launching the kernels. In batches MLX evaluates 3,626 waveforms per second (at 1024 binaries per call), 26.0× JAX's best on the CPU (139 per second), and 58 gradients per second (at 256), 5.4× JAX's best (11). The gradient is a Jacobian along twelve directions, each thread recomputing the values: at large batches its time goes to the PNR angles on the grid of the interpolants and to the angles and modes at the frequencies. In a new process, importing takes 502 ms, the first waveform 16.7 s (tracing the stages, lowering them and compiling their kernels) and the first gradient 30.2 s (the JVP kernels); JAX compiles the model in 11.9 s and its gradient in 47.4 s.

4 IMRPhenomXAS

IMRPhenomXAS, the aligned-spin (2,2) mode of IMRPhenomX, is a public approximant (`"IMRPhenomXAS"`) and has an MLX version (`waveformx.models.xas.mlx`). Both are written for the cases it is mostly called in: non-uniform grids of up to about a thousand frequencies, often many binaries at once, and long uniform grids.

A table per binary, a short formula per frequency

The coefficients come from IMRPhenomXPNR's (2,2) mode (`models/xpnr/internals.py`, `amp22.py`, `phase22.py`), so the fits, collocation systems and connections are LAL's. `models/xas/core.py` then rewrites the model in powers of the frequency in Hz. In each of the three regions of the phase, LAL's sum of powers of Mf and their products with log Mf becomes a sum over 16 powers of f (at most 13 per region), with the powers of M folded into the coefficients and $\log Mf = \log M + \log f$. LAL's time shift and its reference phase (but for the part of the leading term, below) fold into the same coefficients. The amplitude becomes a polynomial in $f^{1/3}$, a rational function and an exponential. Each binary then has a table of 86 numbers, and a frequency needs only its own powers. In a batch with shared frequencies (`jax.vmap`) those powers are computed once, and per binary and frequency what is left is one region's sum, the arctangent and exponential of the merger, and a sine and cosine. The JAX model is one elementwise expression with the regions selected, without the stacked arrays and reductions of a first version, which took twice as long on long grids. Its derivatives are JAX's own, in either mode.

Three changes from LAL's arithmetic make float32 and float64 compute the same thing. Both work in solar masses rather than LAL's round trip through kilograms. LAL's quasi-normal-mode fits are ratios of polynomials in the final spin whose numerator and denominator both nearly vanish at $a_f = 1$. In float32 that cancellation loses a part in 10^4 of the ringdown frequency at $a_f = 0.9$ and up to 1% near 1, so the fits are evaluated in powers of $1 - a_f$, with coefficients re-expanded in float64. In float64 they differ from LAL's form by up to 10^{-11}, LAL's own rounding. The third change, the leading term of the phase, is described below. One fix reaches IMRPhenomXPNR too: the peak frequency of the ringdown amplitude took sqrt($1 - \gamma_2^2$) in the branch not taken for $\gamma_2 > 1$. That gave NaN reverse-mode gradients (XPNR's forward-mode Jacobians did not see it), and the square root is now guarded as the others are, in `scripts/generate_fits.py`.

float32 and binary neutron stars

From 20 Hz the inspiral phase of a binary neutron star reaches ~10^4 radians, where one float32 ulp is ~10^{-3} radians. Summing the phase in float32 lost about ten ulps, from the rounding of the coefficient of $f^{-5/3}$, of the reference phase that cancels it and of the power itself. Light binaries then had mismatches of 10^{-5}–10^{-4} against float64. The leading term is now separate. Its coefficient is $T_0 = K_0\, M^{1/3} / (m_1\, m_2)$, its value at the reference frequency is A, and $T_0\, f^{-5/3} - A$, its sum with the rest and the reduction to $[-\pi, \pi]$ are computed in extended precision. JAX uses float64, which the package always enables; Metal uses double-float pairs of floats, with sums and products made exact by fused multiply-adds. Every other term stays below ~10^3 radians and is computed in the working precision, as are the derivatives.

Against float64 at the same float32-rounded inputs, the weighted mismatch (aLIGO design, the larger of the polarizations) over 300 random binaries of 2–500 $M_\odot$ from 20 Hz has median 9.4×10^{-10} and maximum 1.3×10^{-7} in JAX's float32, and 1.2×10^{-9} and 3.2×10^{-7} in MLX. Over 100 binary neutron stars (1–2.5 $M_\odot$, spins up to 0.6) on 1000 frequencies from 20 Hz, the maxima are 3.4×10^{-8} (JAX) and 7.4×10^{-8} (MLX). On a uniform grid of 1/256 Hz to 2048 Hz (half a million frequencies; 10 binaries) they are 1.8×10^{-8} and 2.5×10^{-8}. The gradient of the log-likelihood of `tests/test_xpnr_float32.py` is within that test's bound for 29 of 30 binaries in JAX's float32 (at most 1.0×10^{0} of the bound) and 30 in MLX (at most 7.6×10^{-1}).

In float64, over 300 random binaries (2–500 $M_\odot$, f_{min} of 10–20 Hz, 1000 frequencies), the model agrees with `SimInspiralChooseFDWaveformSequence` to 2.4×10^{-10} at most (median 8.8×10^{-13}), with mismatches below 4.4×10^{-16}.

On Apple GPUs with MLX

A call is two Metal kernels. The set-up is generated from the JAX function, as for IMRPhenomXPNR (the lowering of `models/xpnr/mlx/_program.py`): one thread per binary, and one per binary and parameter for its forward-mode derivatives. The double-float leading term is written by hand into the same kernel. The waveform at the frequencies is written by hand, one thread per binary and frequency, with the region chosen by branching. Its derivatives along the eight parameters of an aligned-spin binary are linear in those of the table, so they cost a few multiply-adds per direction. `polarizations_and_jacobian` writes them out. For `mx.grad`, a third kernel contracts them with the cotangents and sums over the frequencies inside the kernel, so the Jacobian is never stored.

The set-up of one binary runs on one GPU thread, where a precise division takes ~240 ns and an exponential ~500 ns. It uses Metal's fast division, exponential and logarithm, which halves its time without changing the accuracy above. With MLX's ~0.2 ms to submit and wait for the GPU, one binary takes 482 µs, against 150 µs for JAX on the CPU and 84 µs for LALSimulation. MLX pays off with size. One binary on 2^{20} frequencies takes 566 µs (16 binaries 1.72 ms), and batches reach 5,098,279 waveforms of 1000 frequencies per second at 4096 binaries per call, 17.6× JAX's best on the CPU. In a new process, importing takes 528 ms, the first waveform 607 ms (tracing the set-up and compiling the kernels) and the first gradient 20.18 ms.

	JAX FLOAT64	JAX FLOAT32	JAX, LAL'S EVALUATION	LALSIMULATION	MLX
One binary, 1000 frequencies	150 µs	127 µs	214 µs	84 µs	482 µs
One binary, 2^{20} uniform frequencies	5.75 ms	5.02 ms	13.40 ms	49.39 ms	566 µs
256 binaries × 1000 frequencies, per binary	6.35 µs	4.93 µs	8.96 µs	84 µs	1.86 µs
4096 binaries × 1000 frequencies, per binary	4.63 µs	3.45 µs	8.31 µs	84 µs	0.20 µs
Gradient, one binary, 1000 frequencies	537 µs	621 µs	540 µs	–	1.19 ms
Gradient, one binary, 2^{20} frequencies	–	–	–	–	1.81 ms

Median times of IMRPhenomXAS (`report/xas.py`; data `report/data/xas.json`). The 1000 frequencies are geometric from 20 to 1024 Hz; the 2^{20} are uniform from 20 Hz in steps of 1/512 Hz. Batches are random binaries of 5–200 $M_\odot$, under `jax.jit(jax.vmap(...))` in JAX and as one call in MLX (Apple M2 Ultra GPU). "JAX, LAL's evaluation" is the previous implementation, IMRPhenomXPNR's (2,2) mode evaluated as LAL does, region by region in *Mf*. LALSimulation runs on one thread, one binary at a time. The gradient is that of the power of h_+.

On the CPU, the table makes long grids and batches cheaper. One binary on 2^{20} frequencies takes 5.75 ms in float64 and 5.02 ms in float32, against 13.40 ms for LAL's evaluation in JAX and 49.39 ms for LALSimulation. Per binary, batches of 4096 are 1.8× faster than LAL's evaluation. For one binary on 1000 frequencies (150 µs) the set-up dominates: about 300 XLA fusions of scalar arithmetic, which LAL's compiled C (84 µs) does not have to launch. Removing the optimization barriers that keep these fusions apart made the set-up faster to run but slower to compile (up to eight times for its gradient), so they stay. The model compiles in 741 ms, under `jax.vmap` in 1.0 s and its gradient in 3.1 s (LAL's evaluation: 690 ms, 1.0 s and 2.6 s).

5 IMRPhenomXHM

IMRPhenomXHM, IMRPhenomXAS with the (2,|1|), (3,|3|), (3,|2|) and (4,|4|) modes, is a public approximant (`"IMRPhenomXHM"`) with an MLX version (`waveformx.models.xhm.mlx`), written as IMRPhenomXAS is: tables per binary, a short formula per frequency. It returns the spherical-harmonic modes as well as the polarizations.

One code path

Where LALSimulation offers several versions, only its defaults are implemented: the 122022 release with recalibrated amplitudes (`PhenomXHMReleaseVersion`; IMRPhenomXPNR uses 122019), the default inspiral, intermediate and ringdown versions of each mode, and no multibanding (LAL's default for uniform grids,

which interpolates the modes to ~10^{-4}). The modes are those of the polarizations (`IMRPhenomXHM_MultiMode2`): the (3,2) mode recycles the (2,2) mode in its ringdown and the higher modes are not cut at 0.3/M, where LAL's one-mode functions differ. Mode arrays are not supported. The 122022 amplitudes are new code (their fits generated from waveformrs into `_generated/hm_amp_fits_*.py`); the phases reuse the higher-mode code of IMRPhenomXPNR (`models/xhm/reference.py` is LAL's computation in *Mf*, `core.py` the tables).

Four changes from LAL keep float32 and float64 one computation. The derivatives LAL takes by finite differences (the inspiral and ringdown amplitudes at the boundaries of the intermediate region, with a step of 10^{-9}; the first and second derivatives of the rotated (3,2) ringdown phase that fix its intermediate phase, with a step of 10^{-7}) are exact. LAL's second difference amplifies rounding by ~10^{14}, so its (3,2) differs from the exact one by a mismatch of 8.0×10^{-10} (median; at most 5.5×10^{-6}); waveformrs reproduces LAL's values bit for bit, so they are LAL's. The QNM and mixing fits of the higher modes are in powers of $1 - a_f$, as for the (2,2), and the mixing coefficients are kept in the working precision (LAL rounds them to single precision, which made the waveform a staircase in the parameters). Two ill-conditioned collocation systems, the intermediate amplitude (up to eight coefficients) and the intermediate phase of the (3,2) (a quartic in 1/f), are solved for the same polynomials in variables centred on their regions. The pseudo-PN amplitude coefficients come from the inverse of their constant system rather than LAL's closed form, whose small products underflow in the derivatives in float32.

The phase of every mode is that of the (2,2) rescaled in the inspiral, aligned with it at low frequency, and in the ringdown of the (3,2) a rotation of the (2,2) and the spheroidal (3,2). The large reference phase of the (2,2), ~10^4 radians for binary neutron stars, enters all of them through that alignment and rotation; it is taken out exactly ($(m/2)\,T_0\,(2f/m)^{-5/3}$ is the leading term of mode m), so each mode is $(m/2)^{8/3}\,T_0\,f^{-5/3} - (m/2)$ A plus a phase of at most ~10^3 radians, and the first part is computed in extended precision as for the (2,2).

Accuracy

In float64, over 200 random binaries (as for IMRPhenomXAS, frequencies below 0.3/M), the polarizations agree with `SimInspiralChooseFDWaveformSequence` to 2.1×10^{-4} at most (median 1.5×10^{-7}) and mismatches below 5.1×10^{-8}, the difference being the (3,2) mode's; the (2,2), (2,1), (3,3) and (4,4) modes agree with LAL's one-mode functions to 3.0×10^{-8}. In float32, against float64 at the same inputs, the weighted mismatch of the polarizations over 200 random binaries from 20 Hz is at most 1.3×10^{-7} in JAX and 3.2×10^{-7} in MLX, and of the worst mode 5.6×10^{-7} and 1.3×10^{-6}; over 60 binary neutron stars at most 3.3×10^{-8} and 7.3×10^{-8} (worst mode 1.9×10^{-7}, 3.5×10^{-7}), and on a uniform grid of 1/256 Hz 1.9×10^{-9} and 3.4×10^{-9}. Near maximal spin the (3,2) mode alone loses more in float32 (~10^{-4} at $q = 25$, $\chi = 0.995$): the second derivative of the (2,2) ringdown phase that fixes its intermediate phase comes from IMRPhenomXPNR's collocation of the (2,2) ringdown, ill-conditioned in float32. The gradient of the log-likelihood of `tests/test_xpnr_float32.py` is within that test's bound for 20 of 20 binaries in JAX's float32 (at most 1.5×10^{-1} of it) and 20 in MLX (at most 2.1×10^{-1}).

The mass-scaled reference

IMRPhenomXAS and IMRPhenomXHM take `reference_mf` as the other models do: `phi_ref` is given at f_{ref} = `reference_mf`/M, `Binary.f_ref` must be zero (NaN otherwise, with a zero derivative with respect to it). The models replace f_{ref} by `reference_mf`/M before the table is made, so its dependence on the masses enters the derivatives; in MLX the set-up kernels take it as their input, and the Jacobian adds its column to those of the masses. The converters (`models/xas/reference.py`, for both models, and in MLX lowered from them) move `phi_ref` by half the change of the (2,2) phase (LAL's `phase_22/η + linb Mf`) between the references. IMRPhenomXHM aligns its higher modes with the (2,2) at the reference, so mode m turns by m/2 times that change: `phi_ref` is kept modulo 2π, not π, which would negate the (2,1) and (3,3) modes. The converted binary gives every mode and the polarizations to rounding (largest relative difference ~10^{-12} for binary neutron stars, `tests/test_xas.py`, `tests/test_xhm.py`), and for aligned spins IMRPhenomXPNR's converters give the same `phi_ref` to 10^{-8}.

On Apple GPUs with MLX

The set-up is five generated kernels run side by side, one for the (2,2) and the factors of the modes, one per higher mode. The waveform at the frequencies is a kernel written by hand that calls a function generated from each mode's formula, with the leading terms in double-float. The Jacobian takes the forward-mode derivatives of the tables and of these functions, eight directions at once. The gradient is reverse mode: the cotangent of each mode's table summed over the frequencies in the kernel, then contracted with the table's Jacobian. That costs about ten times the waveform, where forward mode cost fifty times. One binary takes 1.22 ms (JAX 520 µs, LALSimulation 325 µs), 2^{20} frequencies 2.78 ms, and batches reach 543,075 waveforms of 1000 frequencies per second at 4096 binaries per call (12.1× JAX's best on the CPU), and a gradient 20 µs per binary in a batch of 4096. In a new process the first waveform takes 5.4 s and the first gradient 211 ms.

	JAX FLOAT64	JAX FLOAT32	LALSIMULATION	MLX
One binary, 1000 frequencies	520 µs	464 µs	325 µs	1.22 ms
One binary, 2^{20} uniform frequencies	37.76 ms	19.07 ms	393 ms	2.78 ms
256 binaries × 1000 frequencies, per binary	43 µs	31 µs	325 µs	6.23 µs
4096 binaries × 1000 frequencies, per binary	38 µs	22 µs	325 µs	1.84 µs
Gradient, one binary, 1000 frequencies	2.89 ms	4.30 ms	–	3.83 ms
Gradient, 4096 binaries × 1000, per binary	–	–	–	20 µs
Gradient, one binary, 2^{20} frequencies	–	–	–	22.50 ms

Median times of IMRPhenomXHM (`report/xhm.py` ; data `report/data/xhm.json`), on the frequencies and binaries of the IMRPhenomXAS table. LALSimulation runs on one thread, one binary at a time.

On the CPU one binary on 2^{20} frequencies takes 37.76 ms in float64 and 19.07 ms in float32, 10.4× faster than LALSimulation (393 ms). The model compiles in 5.3 s and under `jax.vmap` in 5.8 s; its gradient takes 81.9 s to compile, most of it the reverse mode through the set-up of the (3,2), whose phase needs the second derivative of its rotated ringdown (a forward-mode Jacobian of the tables halves that, at the cost of a slower gradient).

6 TaylorF2 and IMRPhenomD

TaylorF2 and IMRPhenomD are public approximants (`"TaylorF2"` , `"IMRPhenomD"`) with MLX versions (`waveformx.models.taylorf2.mlx` , `waveformx.models.phenomd.mlx`), written as IMRPhenomXAS is: a table of coefficients per binary, a short formula per frequency, the leading term of the inspiral phase in extended precision. They are ported from waveformrs (`src/taylor.rs` , `src/pn.rs` , `src/imrphenomd/`) and checked against LAL.

TaylorF2 has LAL's defaults: the 3.5PN phase with every aligned-spin term, the leading-order amplitude and no tidal terms. Its phase is LAL's sum over powers of $v = (\pi M f)^{1/3}$ and their products with $\log v$, written in powers of f in Hz (eight terms but for the leading one) with the powers of M folded into a table of 15 numbers. Its post-Newtonian phasing (`models/taylorf2/pn.py`) is shared with IMRPhenomD. As in LAL's `SimInspiralChooseFDWaveformSequence` , every frequency is evaluated, without the cut at the innermost stable circular orbit of LAL's uniform grid, and `f_ref = 0` refers the phase to coalescence. Over 300 random binaries (as for IMRPhenomXAS, to the Schwarzschild ISCO or 2048 Hz) it agrees with LAL to 3.8×10^{-10} at most (median 1.6×10^{-12}), mismatches below 4.4×10^{-16}: rounding, mostly of the ~10^4 radians of the phase of light binaries.

IMRPhenomD has three regions each in its phase and amplitude. The inspiral phase is TaylorF2's (without the 3PN spin-spin term, which IMRPhenomD was not calibrated with) and four pseudo-PN terms; the intermediate phase is a sum of powers and a logarithm; the merger-ringdown phase adds an arctangent. The amplitude is a polynomial in $f^{1/3}$, a quartic and a Lorentzian times an exponential. The table has 75 numbers. Three things differ from LAL. The masses are in solar masses and are not reordered (the fits are symmetric under the exchange of the bodies), so the waveform is smooth through equal masses. The quartic of the intermediate amplitude is solved for in powers of $Mf/Mf_{\rm peak}$, better conditioned in float32. And the quasi-normal-mode frequencies come from closed-form fits (below). Where LAL drops a square root of the peak frequency ($\gamma_2 > 1$), its argument is kept positive in the branch not taken, so that the derivatives stay finite.

The quasi-normal-mode frequencies

LAL interpolates the ringdown and damping frequencies of the (2,2) mode in the final spin a with a natural cubic spline through 1003 tabulated values (`QNMData_a` , `QNMData_fring` , `QNMData_fdamp` , given to six or seven digits). waveformx replaces the spline with Chebyshev series of degree 20 in $((1-a)/2)^{1/3}$: the frequencies vary like powers of $1-a$ near $a = 1$, where the damping frequency vanishes, and the cube root makes them smooth. Rational functions in the same variable and in $(1-a)^{1/2}$ were tried too; their fits were less accurate and some had poles in the range. `scripts/fit_phenomd_qnm.py` fits the series to the relative error, with weights reduced for points far off the others, and writes `models/phenomd/_qnm_fit.py` . The error of LAL's spline is estimated by leaving out each point of the table and evaluating the spline through the others there. That is the interpolation error at twice the spacing, an overestimate where the frequencies are smooth, together with the noise of the table, which the spline passes through.

	FINAL SPIN	FIT, MEDIAN	FIT, MAX	SPLINE LEFT OUT, MEDIAN	SPLINE LEFT OUT, MAX	FIT − SPLINE, MEDIAN	FIT − SPLINE, MAX
Mf_{ring}	[-1.00, -0.90]	6.5×10^{-7}	5.1×10^{-5}	1.1×10^{-6}	5.4×10^{-5}	6.0×10^{-7}	3.1×10^{-5}
Mf_{ring}	[-0.90, +0.50]	4.3×10^{-7}	1.1×10^{-6}	5.2×10^{-7}	2.2×10^{-6}	3.2×10^{-7}	1.2×10^{-6}
Mf_{ring}	[+0.50, +0.90]	3.2×10^{-7}	6.2×10^{-6}	3.6×10^{-7}	1.1×10^{-5}	2.5×10^{-7}	4.7×10^{-6}
Mf_{ring}	[+0.90, +0.99]	1.7×10^{-6}	4.9×10^{-6}	3.1×10^{-6}	8.0×10^{-6}	1.2×10^{-6}	4.5×10^{-6}
Mf_{ring}	[+0.99, +1.00]	2.8×10^{-7}	1.1×10^{-6}	8.1×10^{-4}	7.1×10^{-3}	4.1×10^{-4}	4.1×10^{-3}
Mf_{damp}	[-1.00, -0.90]	1.9×10^{-6}	3.8×10^{-4}	1.9×10^{-6}	2.9×10^{-4}	1.5×10^{-6}	3.2×10^{-4}
Mf_{damp}	[-0.90, +0.50]	1.9×10^{-6}	3.8×10^{-6}	2.2×10^{-6}	8.0×10^{-6}	1.3×10^{-6}	4.7×10^{-6}
Mf_{damp}	[+0.50, +0.90]	2.1×10^{-6}	5.0×10^{-6}	2.6×10^{-6}	9.0×10^{-6}	1.4×10^{-6}	5.6×10^{-6}
Mf_{damp}	[+0.90, +0.99]	3.2×10^{-7}	3.8×10^{-6}	8.9×10^{-7}	4.8×10^{-5}	2.8×10^{-7}	2.2×10^{-5}
Mf_{damp}	[+0.99, +1.00]	3.8×10^{-8}	5.0×10^{-7}	8.6×10^{-3}	1.6×10^{-1}	5.1×10^{-3}	1.5×10^{-1}

Relative errors of the QNM fits (`report/data/phenomd_qnm.json`): the fit against the table, the spline through the table without each point against that point, and the fit against LAL's spline halfway between the points.

For a between −0.9 and 1 the fit's error is no larger than the spline's, median or largest. Between −0.9 and 0.99 it is the rounding of the table (typically 1.9×10^{-6}), and the fit and LAL's spline differ by at most 2.2×10^{-5}. Above 0.99, where the damping frequency falls steeply towards zero, the spline left out errs by up to 16% and differs from the fit by up to 15% between the last points, while the fit follows the table to 1.1×10^{-6}. Below −0.9 the table has steps (its first values come from a separate computation): the spline follows them and the fit passes between them, so that there the damping frequency's largest error, 3.8×10^{-4}, exceeds the spline's left-out error (2.9×10^{-4}). Final spins below −0.9 need the larger spin near −1 and mass ratios above about 60, beyond those IMRPhenomD was calibrated to (18). With LAL's spline in place of the fits, the model agrees with LAL to rounding (`tests/test_phenomd.py`); with the fits, over 300 random binaries (to Mf = 0.2 or 2048 Hz), the mismatch with LAL is at most 6.6×10^{-10} (median 2.9×10^{-13}), and the relative error at most 5.0×10^{-5}, near the merger. The fits are smooth in the spin, unlike the spline's second derivative, and cost two Clenshaw sums per binary.

The mass-scaled reference. Both models take `reference_mf`, as the lean IMRPhenomXPNR does: `phi_ref` is then given at Mf_{ref} = `reference_mf` rather than at `Binary.f_ref`, which must be zero (NaN otherwise, with a zero derivative). For these aligned-spin (2,2) models the reference only fixes the constant of the phase, which the tables compute in Mf directly, and the leading term's part in float64 (double-float in MLX) from the masses. `to_mass_scaled_reference` and `to_physical_reference` (`models/taylorf2/reference.py`, `models/phenomd/reference.py`, and in MLX lowered from the JAX functions) change `phi_ref` by half the change of the model's phase between the two references (for IMRPhenomD including its time shift), reduced modulo π; the converted binary gives the same waveform to rounding (`tests/test_taylorf2.py`, `tests/test_phenomd.py`). The JAX converters compute the phase in float64; in MLX's float32 the ~10^4 radians between the references of a binary neutron star cost ~10^{-3} radians of `phi_ref`.

Single precision and derivatives

Against float64 at the same float32-rounded inputs, over 300 random binaries from 20 Hz, TaylorF2's weighted mismatch is at most 1.5×10^{-7} in JAX's float32 and 3.0×10^{-7} in MLX; over 100 binary neutron stars on 1000 frequencies 3.9×10^{-8} and 7.2×10^{-8}, and on a uniform grid of 1/256 Hz to the ISCO 8.5×10^{-9} and 2.8×10^{-8}. In IMRPhenomD the connection of the inspiral phase to the intermediate one needs the inspiral's derivative at Mf = 0.018, a sum of terms of ~10^4 that cancel to ~10^2. In float32 that moves the merger in time by up to tens of microseconds for binaries of a few hundred solar masses. JAX therefore computes each binary's table in float64 and the waveform at the frequencies in float32. Its mismatch with float64 is then at most 2.6×10^{-8} over 300 random binaries and 6.9×10^{-9} over binary neutron stars. MLX has no float64 on the GPU and computes the table in float32, so it reaches 5.3×10^{-6} for heavy binaries (6.7×10^{-8} for binary neutron stars, 2.8×10^{-8} on the uniform grid). The gradient of the log-likelihood of `tests/test_xpnr_float32.py` is within that test's bound for 30 of 30 TaylorF2 binaries in JAX's float32 and 30 in MLX; for IMRPhenomD for 30 and 26 of 30 (at most 8.1×10^{-1} and 2.7×10^{0} of the bound).

Cost

The MLX versions share `models/taylorf2/mlx/_tables.py`, which generalises IMRPhenomXAS's kernels. The set-up kernel is generated from the JAX table and patched with the double-float leading term, the waveform at each frequency is written by hand, and a VJP kernel serves `mx.grad`. One binary on 1000 frequencies takes 70 µs (TaylorF2) and 128 µs (IMRPhenomD) in JAX on the CPU, against 52 µs and 114 µs for LALSimulation; on 2^{20} frequencies 6.65 ms and 5.44 ms, 7.4× and 12.5× faster than LAL. In MLX, batches reach 5,915,656 waveforms per second for TaylorF2 (4096 binaries per call) and 4,915,199 for IMRPhenomD (4096), against 320 µs and 383 µs for one binary, which MLX's submission to the GPU dominates. JAX compiles TaylorF2 in 127 ms (its gradient in 246 ms) and IMRPhenomD in 405 ms (gradient 1.8 s); in a new process, MLX's first IMRPhenomD waveform takes 242 ms, including the tracing of the set-up and the compilation of the kernels.

TAYLORF2	JAX FLOAT64	JAX FLOAT32	LALSIMULATION	MLX
One binary, 1000 frequencies	70 µs	64 µs	52 µs	320 µs
One binary, 2^{20} uniform frequencies	6.65 ms	3.59 ms	49.15 ms	421 µs
256 binaries × 1000 frequencies, per binary	3.45 µs	2.55 µs	52 µs	1.39 µs
4096 binaries × 1000 frequencies, per binary	2.43 µs	1.23 µs	52 µs	0.17 µs
Gradient, one binary, 1000 frequencies	89 µs	74 µs	–	585 µs
Gradient, 4096 binaries × 1000, per binary	–	–	–	0.46 µs
Jacobian, 4096 binaries × 1000, per binary	–	–	–	0.60 µs
Compilation (first call)	127 ms	136 ms	–	–
Compilation under `jax.vmap`	140 ms	165 ms	–	–
Compilation of the gradient	246 ms	260 ms	–	–

IMRPHENOMD	JAX FLOAT64	JAX FLOAT32	LALSIMULATION	MLX
One binary, 1000 frequencies	128 µs	108 µs	114 µs	383 µs
One binary, 2^{20} uniform frequencies	5.44 ms	4.72 ms	68.12 ms	457 µs
256 binaries × 1000 frequencies, per binary	5.65 µs	4.88 µs	114 µs	1.63 µs
4096 binaries × 1000 frequencies, per binary	4.72 µs	2.89 µs	114 µs	0.20 µs
Gradient, one binary, 1000 frequencies	308 µs	482 µs	–	1.14 ms
Gradient, 4096 binaries × 1000, per binary	–	–	–	0.95 µs
Jacobian, 4096 binaries × 1000, per binary	–	–	–	0.98 µs
Compilation (first call)	405 ms	420 ms	–	–
Compilation under `jax.vmap`	513 ms	536 ms	–	–
Compilation of the gradient	1.8 s	1.9 s	–	–

Median times of TaylorF2 and IMRPhenomD (`report/taylorf2_phenomd.py`; data `report/data/taylorf2.json`, `phenomd.json`), on the frequencies and binaries of the IMRPhenomXAS table (Apple M2 Ultra GPU). LALSimulation runs on one thread, one binary at a time.

7 NRSur7dq4

NRSur7dq4 interpolates numerical-relativity waveforms of precessing binaries. It was trained on mass ratios up to 4 and spin magnitudes up to 0.8, and LAL allows it to extrapolate beyond those. It integrates an ODE for the coprecessing frame, the orbital phase and the spins, with time derivatives given by fits at 233 nodes. It then evaluates the coorbital modes from fits at their empirical-interpolation nodes and rotates them to the inertial frame. The JAX version follows `LALSimIMRPrecessingNRSur.c` and waveformrs's translation of it.

- **Data.** The 17 MB HDF5 file is read once with h5py. Every dynamics fit becomes a dense row over the union of the monomials the fits use, so the nine fits of a node are one matrix–vector product. The fits of the waveform nodes are stored by the order of their mass-ratio monomial, so the mass ratio, which is the same at every node, is folded into the coefficients once per binary.
- **Dynamics.** LAL steps from the reference frequency to the nearest node, takes three RK4 steps (forwards or backwards, depending on that node), and covers the remaining nodes with fourth-order Adams–Bashforth steps in both directions. Since the starting node depends on the binary, the integration is one `lax.scan` of fixed length. It walks backwards from the start and then forwards, evaluating the fits of one node per step as LAL does. The Adams–Bashforth weights depend only on the node and are tabulated once.
- **Splines.** GSL's natural cubic splines solve a tridiagonal system whose matrix depends only on the knots. As in waveformrs, it is factorised once when the data are read, so fitting a spline needs only the two substitution sweeps.
- **Frame rotation.** LAL's Wigner D-matrices use negative powers of the quaternion's components, with special cases where they vanish, as they do for spins along the orbital angular momentum. Here the negative powers are cancelled analytically, which makes each element a polynomial in the quaternion: the same values, no special cases, and finite gradients in the non-precessing limit.
- **Output grid.** The number of samples depends on when the orbital frequency reaches $\pi\, f_{\min}$, so it is found on the host. A compiled pass evaluates the frequency fits, and numpy repeats LAL's search operation by operation. The waveform is then computed on a power-of-two buffer of at least 256 samples, which waveforms of similar length share, and trimmed to the binary's length.

`polarizations_at(binary, times)` evaluates the waveform at any times relative to the peak. It is pure and differentiable with respect to every parameter, including `f_ref`, which moves the reference time and so the frame. It is what `jax.vmap`, `jax.grad` and the conversions use.

The mass-scaled reference

With `reference_mf` the binary is given where the orbital frequency of the surrogate is $M\omega_{ref} = \pi$ `reference_mf`, whatever the mass (`f_ref` zero, NaN otherwise with a zero derivative). The reference time on the dynamics grid then depends on the mass ratio and the spins only, through LAL's interpolation of the frequency fits, which the pure function already traces; no host search is needed and the masses no longer move the reference. `REFERENCE_MF` = 0.01 ($M\omega$ = 0.0314) lies inside the surrogate for every binary it covers: over 24000 binaries with $q \le 6$ and $|\chi| \le 0.8$ the surrogate starts at $M\omega \le 0.0224$ ($q = 6$, both spins 0.8 along L; 0.0202 for $q \le 4$), so values of `reference_mf` down to ~0.0072 work across that range; the public functions raise below the start of a binary's surrogate, and allow a reference below f_{min}.

The converters (`models/nrsur7dq4/reference.py`, and in MLX) integrate the surrogate's dynamics from the old reference as the model does, take the state where the orbital frequency is the new one, and give the spins, the inclination and `phi_ref` in the coorbital frame there, with the angle by which the polarization basis turns (as IMRPhenomXPNR's converters). The time of the new reference follows the model's own rule, the frequency fits at the nodes for the spins at the reference, so it depends on the converted spins. The model starts its integration with an Euler step from the reference to the nearest node, whose first-order error depends on where the reference falls between the nodes (up to a few per cent of the waveform); the converted state is chosen so that this step lands on the old integration's state at that node. Six fixed-point iterations find the time and the state together. What remains is the surrogate's own dependence on where its reference lies: its RK4 start and Adams–Bashforth steps from two different nodes give orbital phases that drift apart by up to ~10^{-2} radians over the waveform. A converted binary therefore gives the same waveform up to the rotation of the polarization basis and that drift: a largest relative difference of 10^{-4}–2×10^{-2} and mismatches, without any maximisation, below 10^{-4}; a round trip, which integrates twice, returns the spins to ~10^{-2} at worst. The converters are pure (`jax.jit`, `jax.vmap`, `jax.grad`, float32); in MLX they agree with JAX float64 to 3×10^{-6} in the spins, with gradients to 10^{-5}.

Single precision and the scale of the output

NRSur7dq4 is evaluated in the precision of the binary's parameters. A `Binary` of float32 arrays is evaluated in float32 throughout, with a float32 copy of the data cast from the float64 arrays (the spline factorisations included). A test checks that no operation of the traced waveform, or of its gradient, is in float64. The public functions take Python numbers, so they stay in float64. Two changes make float32 usable:

- **The output is the strain times 10^{21}.** This applies to every NRSur7dq4 function, the public functions and the conversions included, unless `scaled_strain=False`. The strain itself fits in float32, but its square (~10^{-42}) and a detector's power spectral density (~10^{-47}) do not. The comparisons with LAL in this report use `scaled_strain=False`.
- **Nothing passes through SI units.** LAL converts the masses to kilograms and back (`m * MSUN_SI / MSUN_SI`) and works with the distance in metres. In float32, XLA folded the square of the solar mass in kilograms (~4×10^{60}, beyond float32's range) into the derivative of the mass ratio, so the compiled gradient was NaN. The derivative with respect to a distance of ~10^{25} m underflowed to zero. The masses now stay in solar masses, and the amplitude is one constant times the total mass over the distance in Mpc. In float64 this changes the waveform by rounding only.

What float32 costs in accuracy was measured on 300 pairs of binaries, drawn as for the accuracy comparison (`report/float32_nrsur7dq4.py`). 150 pairs are two independent binaries, whose overlaps are near zero. The other 150 are a binary and a small perturbation of its masses, spins and angles, with a median overlap of 0.95. Both binaries of a pair are evaluated on one time grid, in float64 and in float32, projected on a detector and tapered at the start. Inner products use the aLIGO design sensitivity from 20 Hz, without maximisation, and are computed in float64 unless stated otherwise.

CHANGE FROM FLOAT64 TO FLOAT32	MEDIAN	90TH PERCENTILE	LARGEST
Inner product of the two waveforms, change over the product of their norms	2.4×10^{-6}	2.0×10^{-5}	1.0×10^{-4}
Normalised overlap, independent binaries	9.9×10^{-7}	8.0×10^{-6}	3.9×10^{-5}
Normalised overlap, nearby binaries	4.1×10^{-6}	3.0×10^{-5}	1.0×10^{-4}
Normalised overlap, with the inner products in float32 too	2.1×10^{-6}	1.9×10^{-5}	1.0×10^{-4}
Normalised overlap, with the dynamics in float64	2.8×10^{-7}	1.7×10^{-6}	4.6×10^{-6}
One waveform, noise-weighted relative error $\|\delta h\| / \|h\|$	3.8×10^{-5}	8.3×10^{-5}	2.5×10^{-4}
One waveform, largest relative error $\max\|\delta h\| / \max\|h\|$	5.1×10^{-5}	1.3×10^{-4}	7.5×10^{-4}
One waveform, relative error of $\langle h, h\rangle$	6.4×10^{-7}	3.1×10^{-6}	2.9×10^{-5}
Derivatives of a log-likelihood, relative error (40 nearby pairs)	5.6×10^{-5}	6.0×10^{-4}	4.5×10^{-1}
Derivatives of a log-likelihood, error over the typical size of the component	6.5×10^{-7}	3.1×10^{-5}	2.0×10^{-3}

NRSur7dq4 in float32 against float64. The overlaps and inner products are between the two binaries of a pair. "With the inner products in float32 too" also does the whitened Fourier transforms and sums in float32. "With the dynamics in float64" integrates the ODE in float64 and computes the rest in float32. The log-likelihood is $\langle d, h\rangle - \langle h, h\rangle/2$, with noise-free data d from the other binary; its derivatives exclude f_{ref}, which is zero for half the pairs. Data: `report/data/float32_nrsur7dq4.json`.

An inner product between two different waveforms changes by 2.4×10^{-6} of the product of their norms (median), and by at most 1.0×10^{-4}. Doing the inner products themselves in float32 adds at most 1.7×10^{-7} to the overlap. The error of a single waveform, 3.8×10^{-5} (median, noise-weighted), is mostly rounding noise from the final resampling onto the float32 time grid: with everything up to the polarizations on the surrogate's own times in float64, it is still 2.5×10^{-5}. That noise is almost uncorrelated with another waveform, so it barely moves their inner product. The ODE integrated in float32 is what does. With the

dynamics in float64 and the rest in float32, the change of the overlap falls from 2.1×10^{-6} to 2.8×10^{-7} (median), and from 1.9×10^{-5} to 1.7×10^{-6} at the 90th percentile.

Between nearby binaries the log-likelihood changes by $4.2\times10^{-6}\ \rho^2$ (median; $2.8\times10^{-5}\ \rho^2$ at the 90th percentile), where ρ is the signal-to-noise ratio. That is 1.7×10^{-3} at $\rho = 20$ and 4.2×10^{-2} at $\rho = 100$ (2.8×10^{-1} at the 90th percentile). With noise in the data, a further $\langle n, \delta h\rangle$ of about $\rho\ \|\delta h\| / \|h\|$ is added. The float32 derivatives agree with the float64 ones to 5.6×10^{-5} (median relative error), or 6.0×10^{-4} at the 90th percentile. The largest relative errors are in components whose derivative almost vanishes; measured against the typical size of each component, no error exceeds 2.0×10^{-3}.

On this CPU, float32 is 1.9× as fast for one waveform (2.78 ms against 5.34 ms on 8192 samples). It is 1.8× as fast for a batch of 64 under `jax.vmap` (55.69 ms against 102 ms), and 1.4× for a gradient. The dynamics, a sequence of 230 small steps, take 45% of a float64 batch. Keeping them in float64 would make the inner products about ten times more accurate, at the cost of part of the speed-up on the CPU; the package does not do this. float32 matters most on accelerators, where float64 is much slower or, as on Apple GPUs, missing. The MLX version below runs there.

On Apple GPUs with MLX

`waveformx.models.nrsur7dq4.mlx` is NRSur7dq4 in MLX, Apple's array library, which runs on the GPU of Apple silicon. It is an optional dependency (`pip install waveformx[mlx]`). The module is separate from the JAX code, which it leaves unchanged, and reads the same data through the JAX loader. It provides:

- the pure functions `polarizations_at` and `modes_at` ;
- LAL's waveforms on its grid: `td_polarizations` , `td_modes` and `batch_td_polarizations` ;
- the conditioned waveforms: `simulate_td` , `simulate_fd` and their pure functions.

Everything is float32 and differentiable with `mx.grad` . The pure functions take one binary or a batch along a leading axis. MLX has no float64 on the GPU and no `scan` , `while_loop` or `cond` . Its `vmap` and forward-mode derivatives do not pass through custom kernels. So:

- **Splines as matrices.** The JAX version fits its splines with two substitution sweeps (a `lax.scan`). The knots are fixed, so each spline is a fixed linear map, computed in float64 when the data are read:
  - the spline of the dynamics states, evaluated at the waveform's times, is one 2000 × 230 matrix;
  - the second derivatives of the spline through the polarizations are the inverse of its tridiagonal system applied to the right-hand side.

  For a batch, each map is one matrix product over all the binaries.
- **The dynamics in one Metal kernel.** The 230 sequential RK4 and Adams–Bashforth steps run in a hand-written Metal kernel, with one SIMD group per binary and one state component per lane. A second kernel carries dual numbers through the same steps. It gives the Jacobian of the states with respect to the mass ratio, the spins and the reference time, and the kernel's VJP contracts that Jacobian.
- **The frame rotation in Metal kernels.** The Wigner rotation of every mode sample is a generated kernel. Its VJP is a kernel too: it differentiates with respect to the quaternion through the generators of the rotation, using $D(pq) = D(p)\ D(q)$.
- **Harmonics as Fourier series.** The spin-weighted harmonics are trigonometric series in the inclination, with coefficients computed once in float64. Evaluated directly, the powers of $\cos(\theta/2)$ and $\sin(\theta/2)$ in the gradient were fused by `mx.compile` into a kernel with more arguments than Metal allows.
- **LAL's grids on the host.** The lengths and epochs are computed in float64, as the JAX driver computes them. A batch is set up all at once: the frequency fits of every binary in one call, then LAL's search in numpy over the batch. This takes 2.36 ms for 1024 binaries, against ~0.15 ms per binary one by one. A binary that LAL refuses is set up again on its own, which raises LAL's error. For 1200 binaries the grids agree with the JAX driver's:
  - from 20 Hz: the same length and epoch for 407 of the 407 binaries LAL accepts, and LAL's error for 193 of the 193 it refuses;
  - from the start of the surrogate: the same length and epoch for 600 of the 600 binaries LAL accepts;
  - the start times differ by at most 1.0×10^{-11} M, the rounding of the fits summed in another order.

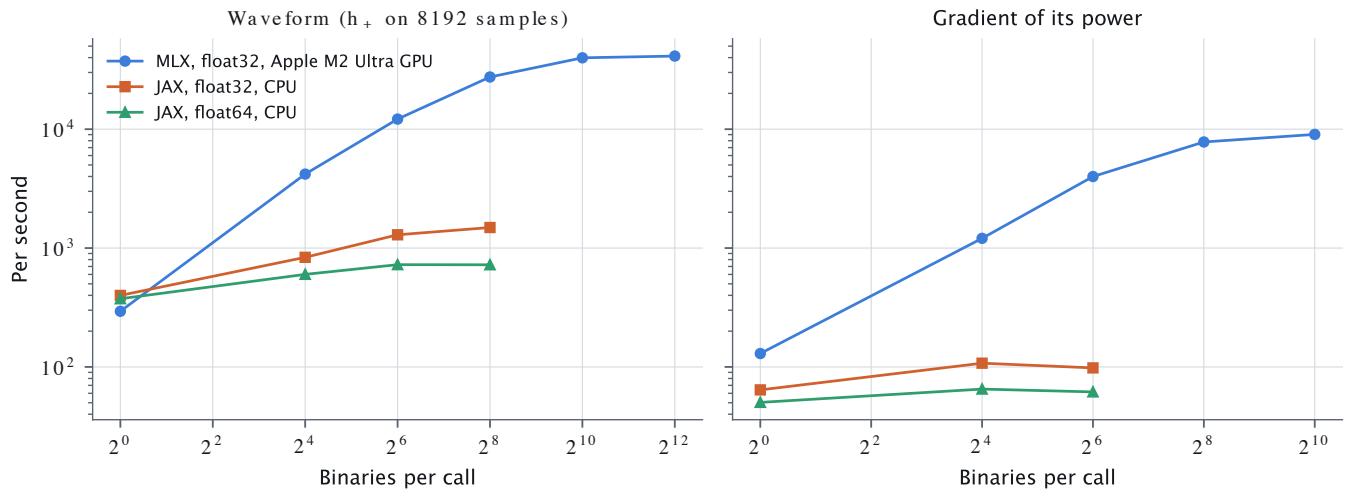
  The binaries of a batch are sorted by length and evaluated in chunks of a power-of-two size.
- **Conditioning.** LAL does not high-pass filter NRSur7dq4, so `SimInspiralTD` and `SimInspiralFD` need only its start taper, the padding and an FFT. The end of the taper depends on the samples, so it is found on the host from the float32 samples.

The accuracy was measured on the same 300 pairs as JAX's float32 above, in the same way (`report/mlx_nrsur7dq4.py`). The MLX waveform is evaluated by `polarizations_at` at the float32 times.

CHANGE FROM FLOAT64 JAX	MLX, FLOAT32			JAX, FLOAT32		
	MEDIAN	90TH PCT.	LARGEST	MEDIAN	90TH PCT.	LARGEST
Normalised overlap of the two binaries, change	2.3×10^{-6}	2.2×10^{-5}	1.2×10^{-4}	2.1×10^{-6}	1.9×10^{-5}	1.0×10^{-4}
Inner product over the product of the norms, change	2.4×10^{-6}	2.2×10^{-5}	1.2×10^{-4}	2.4×10^{-6}	2.0×10^{-5}	1.0×10^{-4}
One waveform, noise-weighted relative error ‖δh‖ / ‖h‖	5.2×10^{-5}	1.1×10^{-4}	2.2×10^{-4}	3.8×10^{-5}	8.3×10^{-5}	2.5×10^{-4}
One waveform, largest relative error max |δh| / max |h|	6.4×10^{-5}	1.6×10^{-4}	7.3×10^{-4}	5.1×10^{-5}	1.3×10^{-4}	7.5×10^{-4}
Derivatives of a log-likelihood, relative error (40 nearby pairs)	5.6×10^{-5}	4.8×10^{-4}	1.3×10^{-1}	5.6×10^{-5}	6.0×10^{-4}	4.5×10^{-1}
Derivatives of a log-likelihood, error over the typical size of the component	8.6×10^{-7}	5.6×10^{-5}	1.1×10^{-3}	6.5×10^{-7}	3.1×10^{-5}	2.0×10^{-3}

NRSur7dq4 in MLX on the GPU and in JAX on the CPU, both in float32, against JAX in float64, on the pairs of the table above. The inner products are computed in float64. JAX's derivatives are those of the table above. Data: `report/data/mlx_nrsur7dq4.json`.

The MLX waveform is as accurate as JAX's float32 one. The overlap of two binaries changes by 2.3×10^{-6} (median; 2.2×10^{-5} at the 90th percentile), against 2.1×10^{-6} (1.9×10^{-5}) in JAX. The derivatives agree with float64 to 5.6×10^{-5} (median relative error). Against the typical size of each component, no error exceeds 1.1×10^{-3}. The conditioned waveforms of 9 cases agree with the JAX conversions to 6.8×10^{-5} of the peak, with the same lengths, epochs and sampling in 9/9.



NRSur7dq4 waveforms (left) and gradients of their power (right) per second, against the number of binaries per call. MLX is on the Apple M2 Ultra GPU, compiled with `mx.compile`. JAX is on the CPU under `jax.jit(jax.vmap(...))`. Each waveform is h_+ on 8192 samples at 4096 Hz, for random binaries.

BINARIES PER CALL	MLX, PER CALL	WAVEFORM, PER BINARY			GRADIENT, PER BINARY		
		MLX	JAX FLOAT32	JAX FLOAT64	MLX	JAX FLOAT32	JAX FLOAT64
1	3.40 ms	3.40 ms	2.50 ms	2.67 ms	7.71 ms	15.60 ms	19.87 ms
16	3.82 ms	239 µs	1.20 ms	1.66 ms	828 µs	9.30 ms	15.36 ms
64	5.26 ms	82 µs	773 µs	1.38 ms	250 µs	10.19 ms	16.22 ms
256	9.32 ms	36 µs	672 µs	1.38 ms	128 µs	–	–
1024	25.70 ms	25 µs	–	–	111 µs	–	–
4096	99.40 ms	24 µs	–	–	–	–	–

Median times of the figure's calls. MLX 0.32.4 runs on the Apple M2 Ultra GPU (64 GB of unified memory); JAX runs on its CPU. The larger JAX batches were not timed.

One waveform takes 3.40 ms in MLX, against 2.50 ms in float32 JAX on the CPU. A single small waveform is dominated by the sequential dynamics and by launching kernels, so the GPU does not help. It pays off in batches:

- **Waveforms.** MLX evaluates 41,208 waveforms per second at 4096 binaries per call. JAX manages at best 1,489 in float32, so MLX is 27.7× faster.
- **Gradients.** One gradient takes 7.71 ms, against 15.60 ms in JAX. In batches MLX computes 9,039 gradients per second, 84.1× JAX's best in float32.
- **The public batch function.** It evaluated 1024 random binaries on LAL's grids, at 4096 Hz from the start of the surrogate (9,210 samples each on average), in 59.75 ms. That is 17,137 waveforms per second, against 564 for JAX's `batch_td_polarizations` (30.4×).

- **One binary.** `td_polarizations` takes 3.43 ms (JAX 3.13 ms), and `simulate_fd` takes 5.82 ms (JAX 3.42 ms).

In a new process, reading the data and preparing its float64 operators takes 1.9 s. The first waveform then takes 39.88 ms and the first gradient 14.52 ms. MLX builds its kernels as they are first used, which costs little here. JAX, by comparison, takes 2.6 s for its first waveform and 10.3 s to compile the gradient.

8 Time and frequency domains

`simulate_td` and `simulate_fd` reproduce LALSimulation 6.2.0's `SimInspiralTD` and `SimInspiralFD`. IMRPhenomXPNR can be given in the time domain and NRSur7dq4 in the frequency domain, and each model can also be conditioned in its own domain. They follow waveformrs's `src/conversions/` and `src/conditioning.rs`.

LAL's conditioning starts the model below the requested frequency by an amount set by bounds on the chirp time, merger time and ringdown time. It tapers the extra length, applies a Butterworth high-pass filter, shifts the waveform in time and crops it to a length and epoch derived from the same bounds. The work is split by what needs to be exact:

- **On the host, in Python floats.** The lengths, shifts, taper widths and GPS epochs are computed with LAL's arithmetic. This includes C's `pow`, `cbrt` and `round`, and the fused multiply-adds of clang's arm64 build, reproduced exactly through rational arithmetic. As a result, lengths and epochs are LAL's bit for bit.
- **In JAX, pure and differentiable.** The bounds are computed again where the waveform depends on them continuously: the starting frequency given to the model and the cutoff of the first high-pass filter. So do the filter coefficients, the filter itself (a `lax.scan` of LAL's recursion per second-order section), the tapers and the Fourier transforms. Arrays have fixed, power-of-two lengths, so one compilation serves many binaries.

IMRPhenomXPNR smooths its antisymmetric amplitude with a moving average over the array of frequencies, which LAL takes over its band. To keep one array length per transform length, the model is evaluated on the band, followed by frequencies just above it and then the band's last 82 frequencies again. The first L − 41 values are then averaged as LAL averages them, and the last 41 are taken from the end. `simulate_td_function` and `simulate_fd_function` return the conversion as a pure function. It keeps the lengths, shifts, tapers and epoch of a reference binary, while the starting frequencies and filter cutoffs follow the binary, so the gradient includes their dependence on the masses and spins.

9 How the package is tested

818 pytest tests (816 passed, 0 failed, 2 skipped in the run below) compare the models with LALSimulation 6.2.0, through its Python API or, for internal functions, through ctypes, and with waveformrs, which reproduces LALSimulation with the parity fix of IMRPhenomXPNR. They also reuse waveformrs's reference suites, which cover each regime switch and fallback of the models.

TEST FILE	TESTS	RESULT	TIME
`test_quick.py` The quick tier (`pytest`): every model once under `jit` , `vmap` and a derivative, against LAL; the conversions; the matches	11	11 passed	41.1 s
`test_api.py` The interface: options, arguments, errors, spherical harmonics	12	12 passed	4.1 s
`test_xpnr_xas.py` IMRPhenomXPNR's (2,2) mode without precession against `SimIMRPhenomXASFrequencySequence`	11	11 passed	307 ms
`test_xas.py` IMRPhenomXAS against LAL through the public functions, float32 (binary neutron stars too), batches, LAL's errors, the QNM fits	25	25 passed	60.7 s
`test_transforms_xas.py` IMRPhenomXAS under `jit` , `vmap` , `grad` and `jvp` , in float64 and float32	9	9 passed	43.3 s
`test_xhm.py` IMRPhenomXHM against LAL (polarizations, one-mode functions, `ChooseFDModes`), the exact derivatives of the (3,2), float32, batches, errors	14	14 passed	127.5 s
`test_transforms_xhm.py` IMRPhenomXHM under `jit` , `vmap` , `grad` and `jvp` , in float64 and float32, at equal masses	9	9 passed	184.3 s
`test_xhm_mlx.py` IMRPhenomXHM in MLX against JAX: polarizations and modes on non-uniform and uniform grids, Jacobian, gradients, batches	15	15 passed	72.9 s
`test_xas_mlx.py` IMRPhenomXAS in MLX against JAX and LAL: waveform on non-uniform and large uniform grids, Jacobian, gradients, batches	24	24 passed	25.5 s
`test_taylorf2.py` TaylorF2 against LAL and waveformrs through the public functions, float32 (binary neutron stars too), batches, errors	24	24 passed	4.7 s
`test_transforms_taylorf2.py` TaylorF2 under `jit` , `vmap` , `grad` and `jvp` , in float64 and float32	9	9 passed	4.3 s
`test_taylorf2_mlx.py` TaylorF2 in MLX against JAX and LAL: waveform on non-uniform and large uniform grids, Jacobian, gradients, batches	23	23 passed	4.1 s
`test_phenomd.py` IMRPhenomD against LAL (also with LAL's QNM spline in place of the fits) and waveformrs, the QNM fits, float32, batches, errors	33	33 passed	12.5 s
`test_transforms_phenomd.py` IMRPhenomD under `jit` , `vmap` , `grad` and `jvp` , in float64 and float32	11	11 passed	16.8 s
`test_phenomd_mlx.py` IMRPhenomD in MLX against JAX and LAL: waveform on non-uniform and large uniform grids, Jacobian, gradients, batches	24	24 passed	12.6 s
`test_xpnr_hm.py` Higher modes (release 122019) against `SimIMRPhenomXHMFrequencySequenceOneMode`	24	24 passed	108.7 s
`test_xpnr_msa.py` MSA and NNLO angles and the precession set-up, GSL's elliptic functions	87	87 passed	21.5 s
`test_xpnr_pnr_helpers.py` PNR fits and angle helpers against LAL's internal functions (ctypes)	54	54 passed	18.3 s
`test_xpnr.py` IMRPhenomXPNR polarizations against LAL with the parity fix: waveformrs's reference suite, mirror symmetry	61	61 passed	89.8 s
`test_xpnr_modes.py` IMRPhenomXPNR J-frame modes, `ChooseFDModes` , checks	7	7 passed	29.1 s
`test_xpnr_model.py` IMRPhenomXPNR at special spins, in float32, through the public functions, under `vmap` and `grad`	22	22 passed	211.6 s
`test_xpnr_smooth.py` IMRPhenomXPNR across LAL's switches, the interpolants of the PNR angles, the phase grid, gradients	34	33 passed 1 skipped	105.4 s
`test_xpnr_reference.py` IMRPhenomXPNR with the mass-scaled reference frequency, the converters, invariance, against LAL	39	38 passed 1 skipped	917.8 s
`test_xpnr_float32.py` IMRPhenomXPNR in float32 (no float64 operation, accuracy, gradients) and the scale of its output	6	6 passed	73.5 s
`test_xpnr_random.py` IMRPhenomXPNR, 64 random binaries against LAL with the parity fix (slow)	64	64 passed	17.8 s
`test_transforms_xpnr.py` IMRPhenomXPNR under `jit` , `vmap` , `grad` and `jvp`	5	5 passed	80.6 s
`test_nrsur7dq4.py` NRSur7dq4 polarizations, modes, dynamics, errors, data file, batches	43	43 passed	40.5 s
`test_nrsur7dq4_random.py` NRSur7dq4, 64 random binaries against LAL (slow)	1	1 passed	9.5 s
`test_transforms_nrsur7dq4.py` NRSur7dq4 under `jit` , `vmap` and `grad` , removable singularities	15	15 passed	70.7 s
`test_nrsur7dq4_float32.py` NRSur7dq4 in float32 (no float64 operation, accuracy, `jit` , `vmap` , `grad`) and the scale of its output	14	14 passed	60.3 s
`test_nrsur7dq4_mlx.py` NRSur7dq4 in MLX against JAX: waveform, modes, batches, gradients, the Metal kernels, LAL's grids and the conditioned waveforms	25	25 passed	37.7 s
`test_xpnr_mlx.py` IMRPhenomXPNR in MLX against JAX: the stages, the generated kernels, the SpinTaylor kernel, waveform, Jacobian and gradients	26	26 passed	890.9 s

TEST FILE	TESTS	RESULT	TIME
`test_conversions.py` `simulate_td` and `simulate_fd`: lengths, epochs, filters, tapers against LAL	31	31 passed	29.1 s
`test_transforms_conversions.py` The conversions as pure functions under `jit`, `vmap` and `grad`	6	6 passed	89.4 s

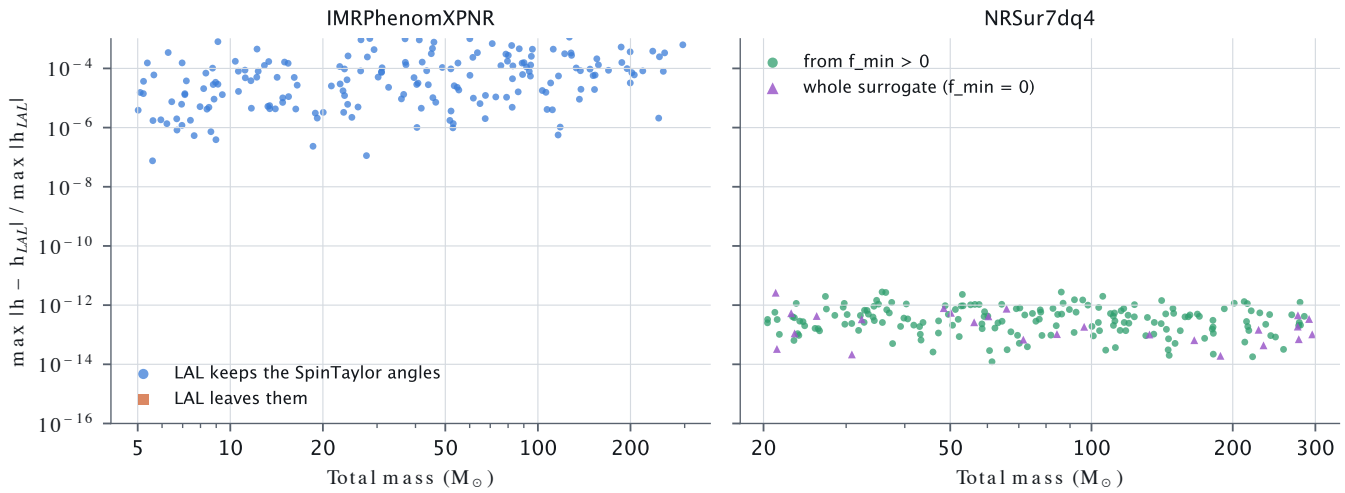
The full run with `WAVEFORMX_SLOW=1` (the random-draw tests) with `OMP_NUM_THREADS=1` for LALSimulation. The MLX tests, which need MLX, come from their own run in a Python with MLX. Data: `report/data/pytest.xml` and `report/data/pytest_mlx.xml`.

The tests are layered as the model was built: the (2,2) mode, then the higher modes, the angles, the PNR helpers, the polarizations and the modes. Each layer is compared at a tolerance set by what LAL itself can resolve there. The MSA angles, for example, are ill-conditioned for nearly aligned spins, where LAL's own output changes by ~5e-8 for a change of the inputs in the last bit. Every model also has a transform test. It checks that `jax.jit` and `jax.vmap` give the plain call, that gradients agree with central finite differences for every parameter, and that they are finite at removable singularities such as aligned, vanishing or single spins.

For IMRPhenomXPNR the layers below the precession angles (the (2,2) mode, the higher modes, the fits and helpers of the PNR angles) are compared with LAL at rounding level, and the polarizations and modes at the model's own tolerances: where LAL uses the SpinTaylor angles, mismatches below 1e-4 (1e-3 beyond q = 12); where it does not, the model must be finite. Further tests check that the model is continuous across LAL's switches and that its derivatives are finite at aligned, single and vanishing spins.

10 Accuracy

The random draws are those of waveformrs's report, so the two reports can be compared. For IMRPhenomXPNR: total masses log-uniform in 5–300 $M_\odot$, mass ratios uniform in 1/20–1, isotropic spins of magnitude up to 0.99, $f_{min} \in \{10, 15, 20\}$ Hz, $f_{ref} = f_{min}$ or up to 3 f_{min}, and isotropic lines of sight. For NRSur7dq4: mass ratios 1–6 with either body first, total masses 20–300 $M_\odot$, sampling at 1–8 kHz, and f_{min} above the surrogate's start or zero.



Largest relative error of each draw against total mass. Left: IMRPhenomXPNR on 4,096 uniformly spaced frequencies from f_{min} to min(0.3/M, 2048 Hz), against LALSimulation 6.2.0 with lalsuite !2593 (through waveformrs), where LAL keeps the SpinTaylor angles and where it leaves them. Right: NRSur7dq4 against `SimInspiralChooseTDWaveform`. Data: `report/data/validation_*.json`.

NRSur7dq4 agrees with LAL to 2.8×10^{-12} at most (median 3.0×10^{-13}), with LAL's length and epoch in every compared draw (200/200). LAL evaluates all 200 draws, and so does waveformx. The residual comes from summing the fits in a different order and from XLA's own `exp`, `sin` and `cos`. The rotation to the inertial frame and the splines are linear, so the errors stay at rounding level. waveformrs, which agrees with LAL bit for bit, gives the same comparison (2.8×10^{-12}).

IMRPhenomXPNR is compared with LAL 6.2.0 with the parity fix through waveformrs, which agrees with the patched LALSimulation to ~3e-15. For the 193 draws for which LAL keeps the SpinTaylor angles, the mismatch has median 2.8×10^{-8} and maximum 1.7×10^{-4} (relative errors: median 5.0×10^{-5}, 90th percentile 5.2×10^{-4}, maximum 9.7×10^{-3}). The difference is the model's: the precession equations solved by a different method and from a lower frequency, LAL's switches made continuous (the two-spin construction for every binary weighs most), exact derivatives in the higher modes. It grows with the total mass, where the merger and ringdown, and so the connection of the angles, carry more of the band: the median mismatch is 2.8×10^{-9} below 20 $M_\odot$ and 1.3×10^{-7} above 150 $M_\odot$.

1% of these draws exceed 1e-4. For the 7 draws for which LAL leaves the SpinTaylor angles (its integration fails, or the in-plane spins are tiny), the model keeps them: the mismatch has median 1.7×10^{-3} and maximum 2.1×10^{-1}. Against the released LAL 6.2.0, without the parity fix, the mismatch where LAL keeps the SpinTaylor angles has median 1.0×10^{-3} and maximum 1.4×10^{-1}. LAL evaluates all 200 draws.

Conversions

FUNCTION	MODEL	BINARY	SETTING	MODES	SAMPLES	GRID	VS LAL	VS WAVEFORMRS	LAL MULTIBANDING	WAVEFORMX	WAVEFORMRS	LAL
`simulate_td`	IMRPhenomXPNR	36+29	4096 Hz from 20 Hz	all	13,006	exact	3.8×10^{-3}	5.5×10^{-4}	8.0×10^{-5}	46.07 ms	7.17 ms	20.59 ms
`simulate_td`	IMRPhenomXPNR	36+29	4096 Hz from 20 Hz	(2,±2)	13,006	exact	3.9×10^{-3}	5.7×10^{-4}	7.4×10^{-5}	29.51 ms	5.08 ms	7.57 ms
`simulate_td`	IMRPhenomXPNR	36+29	2048 Hz from 15 Hz	all	11,142	exact	3.8×10^{-3}	5.7×10^{-4}	4.2×10^{-5}	54.02 ms	29.87 ms	22.57 ms
`simulate_td`	IMRPhenomXPNR	36+29	2048 Hz from 15 Hz	(2,±2)	11,142	exact	3.9×10^{-3}	5.6×10^{-4}	4.0×10^{-5}	40.91 ms	3.35 ms	9.43 ms
`simulate_td`	IMRPhenomXPNR	60+40	2048 Hz from 20 Hz	all	5,398	exact	2.5×10^{-1}	1.7×10^{-4}	1.7×10^{-4}	15.95 ms	3.88 ms	9.02 ms
`simulate_td`	IMRPhenomXPNR	20+12	4096 Hz from 20 Hz	all	26,662	exact	6.3×10^{-2}	8.3×10^{-4}	3.9×10^{-5}	130 ms	30.01 ms	72.09 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf from LAL, 20–2048 Hz	all	8,193	exact	3.8×10^{-4}	6.0×10^{-5}	3.8×10^{-5}	13.98 ms	4.25 ms	9.80 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf from LAL, 20–2048 Hz	(2,±2)	8,193	exact	3.9×10^{-4}	6.0×10^{-5}	3.8×10^{-5}	29.92 ms	1.48 ms	3.55 ms
`simulate_fd`	IMRPhenomXPNR	36+29	Δf = 0.25, 19–1000 Hz	all	4,097	exact	3.7×10^{-4}	5.7×10^{-5}	3.6×10^{-5}	12.87 ms	4.52 ms	10.18 ms
`simulate_fd`	IMRPhenomXPNR	60+40	Δf = 0.125, 15–1024 Hz	all	8,193	exact	2.3×10^{-2}	1.8×10^{-5}	4.0×10^{-5}	13.62 ms	5.32 ms	11.56 ms
`simulate_td`	NRSur7dq4	40+20	4096 Hz from 20 Hz	all	4,225	exact	1.3×10^{-12}	1.3×10^{-12}	–	4.51 ms	1.27 ms	6.89 ms
`simulate_td`	NRSur7dq4	40+20	2048 Hz from 25 Hz	all	1,151	exact	3.1×10^{-13}	3.1×10^{-13}	–	8.31 ms	1.29 ms	7.23 ms
`simulate_td`	NRSur7dq4	70+30	2048 Hz from 14 Hz	all	2,619	exact	2.6×10^{-14}	2.6×10^{-14}	–	6.03 ms	1.23 ms	10.23 ms
`simulate_fd`	NRSur7dq4	40+20	Δf from LAL, 20–2048 Hz	all	4,097	exact	1.1×10^{-13}	1.1×10^{-13}	–	6.90 ms	1.51 ms	7.10 ms
`simulate_fd`	NRSur7dq4	40+20	Δf = 0.25, 19–1000 Hz	all	4,097	exact	8.5×10^{-14}	8.5×10^{-14}	–	7.90 ms	2.22 ms	7.80 ms
`simulate_fd`	NRSur7dq4	70+30	Δf = 0.125, 14–1024 Hz	all	8,193	exact	3.5×10^{-14}	3.5×10^{-14}	–	6.38 ms	1.71 ms	7.85 ms

End-to-end `simulate_td` and `simulate_fd` against `SimInspiralTD` and `SimInspiralFD` with multibanding off. IMRPhenomXPNR has the parity fix, which LAL 6.2.0 does not: its waveforms are compared with waveformrs, LAL 6.2.0 checks the grids. "Grid" says whether the length, epoch and sampling are LAL's exactly. Errors are max $|h - h_{LAL}|$ / max $|h_{LAL}|$ over both polarizations. "LAL multibanding" is how far LAL's default output, with multibanding on, is from the output without it. Times are for one call after the first. Data: `report/data/validation_conversions.json`.

All lengths, epochs and samplings are LAL's (16/16). With LAL's own model in place of the JAX one, the conversions agree with LAL to ~1e-12 of the peak in the time domain and to rounding in the frequency domain (`test_conversions.py`). The time-domain high-pass filter, whose poles are close to z = 1, amplifies the rounding differences of the Fourier transforms, which are not FFTW's, by ~1e4. End to end, NRSur7dq4 agrees with LAL to 1.3×10^{-12}; IMRPhenomXPNR differs from LAL 6.2.0 by its own differences and the parity fix (3.9×10^{-3} for the (2,2) mode, 2.5×10^{-1} for all modes) and agrees with waveformrs as the model does. LALSimulation's default multibanding, which waveformx does not implement, changes LAL's own output by 3.6×10^{-5}–1.7×10^{-4}. This is far more than the differences between the implementations.

11 IMRPhenomXPNR against NRSur7dq4

Where our IMRPhenomXPNR departs from LAL's, the departure is judged against an independent model: NRSur7dq4, interpolated from numerical relativity, at masses where it covers the band. On a fixed set of binaries, if our model's mismatch with NRSur7dq4 is smaller than LAL's, the departure is plausibly an improvement. The set stays fixed, so later changes to the model can be tracked on it.

Matches as pure functions

`waveformx.match` computes matches of frequency series on a common uniform grid with a given PSD, in pure JAX (`jax.jit` , `jax.vmap` , `jax.grad`). The overlap as a function of a time shift comes from a zero-padded inverse FFT; the best sample is refined by Newton steps on the exact overlap, so the result is smooth in the waveforms, and its gradient is that of the overlap at the best time. `mismatch` maximises over time and a constant phase. `polarization_mismatch` maximises the template $F_+h_+ + F_\times h_\times$ over time and the antenna patterns, the norm of the projection of the signal onto the real span of the two polarizations (Harry et al. 2016); the phase is left to the binary, as it is not a free constant for precessing binaries with higher modes. `with_conventions` shifts a binary's reference phase and in-plane spin azimuths, and `optimize` minimises over them from several starts with L-BFGS-B and the gradient through the model. `tests/test_match.py` checks analytic cases (a flat band shifted in time gives the Dirichlet kernel, a constant phase gives cos ϕ, a signal with a part orthogonal to both polarizations gives its projection), the recovery of time and phase shifts off the FFT grid and of any combination of the polarizations, the derivatives against finite differences, and the recovery of shifted conventions of the lean IMRPhenomXPNR.

The comparison

The signal is NRSur7dq4's h_+, through `SimInspiralFD` (LAL's, or our `simulate_fd`); the template is IMRPhenomXPNR at the same binary (LAL's `SimInspiralChooseFDWaveformSequence` , or ours), both with f_{ref} = 20 Hz and LAL's frame conventions, on 20–1024 Hz with Δf = 1/8 Hz and the aLIGO design PSD. The signal is the whole surrogate (f_{min} = 0), tapered by LAL at its start: conditioned from a frequency near the band, its tapered and filtered first cycles fall inside it, which changes the mismatch from 20 Hz by ~3×10^{-3} when conditioned from 20 Hz and ~10^{-4} from 10 Hz, as large as the differences being measured. Binaries are kept only where the surrogate covers the conditioning from 10 Hz. On the benchmark set, the whole surrogate and the surrogate conditioned from 10 Hz differ by at most 4.9×10^{-4}; the whole surrogate starts lower, so what its own start contributes is smaller (from tests at 120–200 $M_\odot$, it falls quickly as the start moves below 10 Hz). Any such error is in the signal, common to LAL's model and ours.

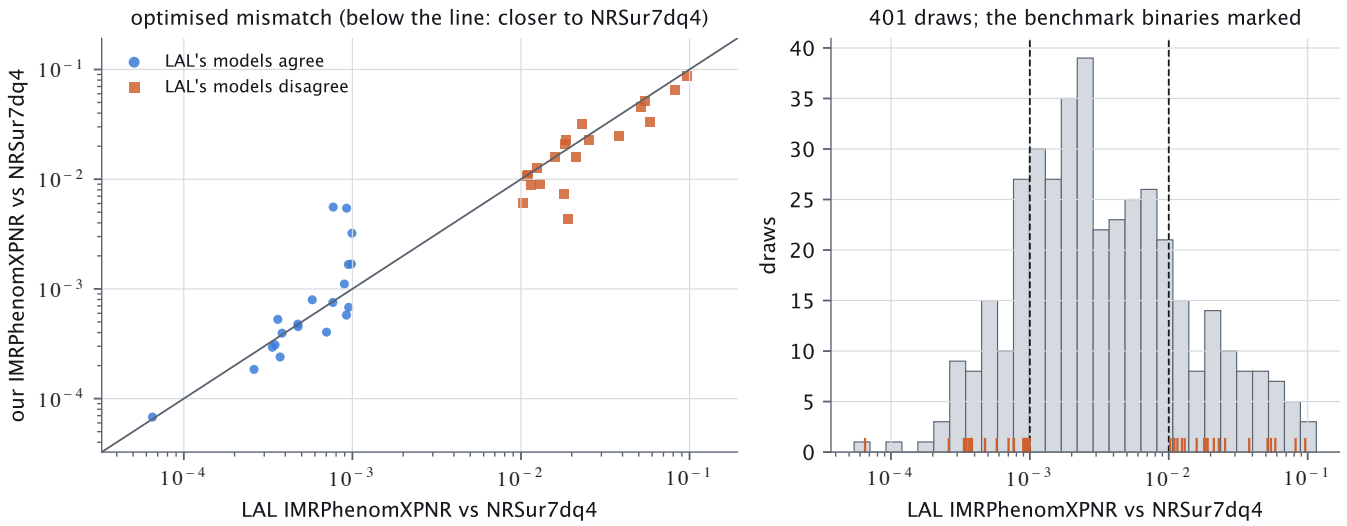
The mismatch is maximised over time and polarisation, then optimised over what the two models define differently: the reference phase and a common rotation of the in-plane spins about the orbital angular momentum, the orientation of the frame in which the spins are given. Without the optimisation the mismatches are much larger (median 1.0×10^{-2} where LAL's models agree); nearly aligned binaries are best at a reference phase shifted by about −π/2, a difference of convention. The angle between the in-plane spins is physical and not optimised: freed, it lets the optimisation reach a different binary. The optimisation takes the best 3 of 16 starts (4 phases times 4 rotations) and runs Nelder–Mead from each, the same for LAL's model and ours: 189 evaluations per optimisation (median), at ~10 ms per model evaluation and ~4 ms per mismatch. The 40 binaries take 106.7 s on 16 processes, much of it compiling the models in each. From the best start alone, Nelder–Mead missed the optimum for a few binaries, by up to a factor of 6.

At the same mass-scaled reference. Both models take `reference_mf` , and they share the orientation conventions: the spins in a frame whose z axis is along the orbit (L_N for IMRPhenomXPNR, the principal axis of the radiation for NRSur7dq4) and whose x axis points along the separation, the line of sight at the azimuth π/2 − `phi_ref` . They do not share the definition of the orbital phase at the reference. NRSur7dq4 sets the orbital phase to zero at the time its orbital frequency is the reference; IMRPhenomXPNR, like every IMRPhenomX model, sets the frequency-domain phase of the (2,2) mode at f_{ref} to 2 `phi_ref` + π/4 with its time origin at the peak. In the stationary-phase picture that phase differs from twice the orbital phase by $2\pi\, f_{ref}\, t(f_{ref})$, with $t(f_{ref})$ the time from the reference to the peak in each model, so the offset between the two descriptions is expected to depend on the binary, as it does. Over 36 aligned-spin binaries (q = 1.25, 2, 3; 100 and 160 $M_\odot$; three spin pairs; two inclinations) at `reference_mf` = 0.01, the mismatch of the same binary in both models is 8×10^{-4} to 0.17 without a shift of `phi_ref` , and 2×10^{-4} to 8×10^{-4} with the best one, which depends on the mass ratio and the spins (from 0.24 to 5.97 radians) but not on the total mass or the inclination (to 0.05 radians). A shared mass-scaled reference therefore removes the frequency convention but not the phase convention: comparisons still need the reference phase optimised, and for precessing binaries the in-plane spin rotation with it.

`report/crossmodel.py --thorough` checks the procedure, once (312.7 s). Nelder–Mead to tighter tolerances (259 evaluations) changes no mismatch by more than 1.4×10^{-3} of itself. The gradient optimisation of `optimize` through our model, from the same starts, agrees with Nelder–Mead to 3.4×10^{-8}. Our NRSur7dq4 as the signal changes the mismatches by 4.9×10^{-12} of themselves.

The benchmark set

500 binaries were drawn in NRSur7dq4's range (mass ratios 1–4, spins up to 0.8 with isotropic directions, total masses 90–200 $M_\odot$, isotropic orientations); 401 have a surrogate covering the conditioning from 10 Hz. LAL's two models agree below 1×10^{-3} for 76 of them and disagree above 1×10^{-2} for 83. From each group 20 binaries were kept, spread over the mass ratio, the total mass, χ_{eff}, χ_p and the inclination by farthest-point sampling (`report/crossmodel.py --select` , 228.8 s on 20 processes; data `report/data/crossmodel_points.json` , with LAL's numbers). The disagreeing binaries have larger mass ratios and in-plane spins.



Left: the optimised mismatch of our IMRPhenomXPNR with LAL's NRSur7dq4 against that of LAL's IMRPhenomXPNR, one point per benchmark binary; below the diagonal our model is closer to NRSur7dq4. Right: LAL's mismatch over the 401 draws, with the thresholds of the two groups (dashed) and the chosen binaries (ticks). Data: `report/data/crossmodel.json`, `crossmodel_points.json`.

SIGNAL	TEMPLATE	LAL'S MODELS AGREE		LAL'S MODELS DISAGREE	
		MEDIAN	MAX	MEDIAN	MAX
NRSur7dq4	LAL IMRPhenomXPNR	6.4×10^{-4}	9.9×10^{-4}	1.9×10^{-2}	9.6×10^{-2}
NRSur7dq4	our IMRPhenomXPNR	5.5×10^{-4}	5.6×10^{-3}	1.8×10^{-2}	8.6×10^{-2}
LAL IMRPhenomXPNR	our IMRPhenomXPNR, same binary*	5.0×10^{-4}	6.6×10^{-3}	1.6×10^{-2}	6.5×10^{-2}

Optimised mismatches on the 40 benchmark binaries, with LAL's NRSur7dq4 as the signal (`report/crossmodel.py`; data `report/data/crossmodel.json`). Our NRSur7dq4 agrees with LAL's to 4.4×10^{-16} on the set. *The last row is not optimised: the two IMRPhenomXPNRs share their conventions, so it is the larger mismatch of the two polarizations at the same binary, maximised over time and phase.

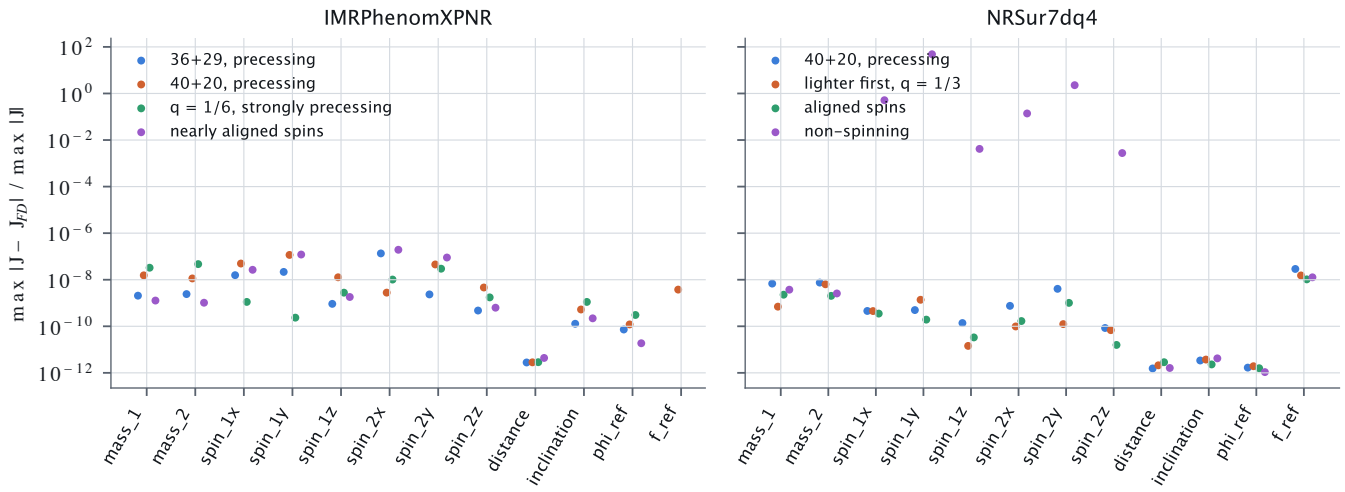
Where our IMRPhenomXPNR sits

Our IMRPhenomXPNR departs from LAL 6.2.0's on this set by a median mismatch of 5.0×10^{-4} where LAL's models agree and 1.6×10^{-2} where they disagree. LAL used its SpinTaylor angles for 40 of the 40 binaries, so the departure is not a different prescription. It is almost all the parity fix (lalsuite !2593, which LALSuite's development branch has and 6.2.0 does not). The model before the break with LAL (tag `lal-faithful`; data `report/data/crossmodel_lal_faithful.json`) still had LAL's model and the fix as an option. Without the fix it agreed with LAL to 4.1×10^{-10}, the fix alone changed it by up to 6.5×10^{-2}, and its lean model differed from it, with the fix, by at most 8.6×10^{-6}. Today's model departs from LAL by the same amounts as that lean model, to 5.0×10^{-6}, so the breakdown still holds. The binaries where LAL's models disagree are those with large in-plane spins and mass ratios, where the fix matters most.

Where LAL's models disagree, our model is closer to NRSur7dq4 for 14 of 20 binaries, with a median ratio of its mismatch to LAL's of 0.89 (from 0.23 to 1.4). Where they agree, it is closer for 9 of 20, median ratio 1.02 (from 0.57 to 7.2): for a few binaries with in-plane spins around 0.4–0.6, the fix moves a waveform that matched NRSur7dq4 to 10^{-3} away from it by a few 10^{-3}. So on balance the parity fix moves IMRPhenomXPNR towards numerical relativity where it is furthest from it. It is not uniformly an improvement. These are 40 binaries at 90–200 $M_\odot$, with NRSur7dq4's own error, not numerical relativity itself, as the reference.

12 Derivatives

For each model and four binaries, every column of the Jacobian (`jax.jacfwd`) is compared with central finite differences of the compiled model. The error of a column is max $|J_i - D_i|$ / max $|J_i|$ over the frequencies or times and both polarizations. The finite difference is the better of a second-order difference with a step of 1e-6 and a fourth-order difference with a step of 1e-4, both relative for the masses, the distance and f_{ref}. The first is limited by rounding, the second by the model's discontinuities.



Disagreement of each column of the Jacobian with finite differences, per parameter and binary. Data: `report/data/gradients.json`.

MODEL	BINARY	MASSES, DISTANCE, ANGLES, F_{REF}	SPINS	WORST PARAMETER	NOT DIFFERENTIABLE	FINITE
IMRPhenomXPNR	36+29, precessing	2.4×10^{-9}	1.3×10^{-7}	`spin_2x`	–	yes
IMRPhenomXPNR	40+20, precessing	1.5×10^{-8}	1.2×10^{-7}	`spin_1y`	–	yes
IMRPhenomXPNR	q = 1/6, strongly precessing	4.7×10^{-8}	3.0×10^{-8}	`mass_2`	–	yes
IMRPhenomXPNR	nearly aligned spins	1.3×10^{-9}	1.9×10^{-7}	`spin_2x`	–	yes
NRSur7dq4	40+20, precessing	2.9×10^{-8}	4.1×10^{-9}	`f_ref`	–	yes
NRSur7dq4	lighter first, q = 1/3	1.5×10^{-8}	1.4×10^{-9}	`f_ref`	–	yes
NRSur7dq4	aligned spins	1.0×10^{-8}	1.0×10^{-9}	`f_ref`	–	yes
NRSur7dq4	non-spinning	1.3×10^{-8}	–	`f_ref`	`spin_1x`, `spin_1y`, `spin_1z`, `spin_2x`, `spin_2y`, `spin_2z` (zero spin)	yes
simulate_td IMRPhenomXPNR (2,2)	36+29, precessing	1.1×10^{-9}	9.8×10^{-9}	`spin_2x`	–	yes
simulate_fd NRSur7dq4	40+20, precessing	2.0×10^{-8}	3.9×10^{-9}	`f_ref`	–	yes

Largest column error by group of parameters. The conversions are `simulate_td_function` of IMRPhenomXPNR's (2,2) mode at 1024 Hz from 25 Hz and `simulate_fd_function` of NRSur7dq4 up to 1024 Hz from 20 Hz.

The derivatives are those of the function JAX computes, which is smooth almost everywhere but not everywhere. Where a model makes a discrete choice that depends on the binary, the waveform is continuous only up to the size of that choice's effect, and the derivative is that of the branch taken: the node where NRSur7dq4's integration starts, and in IMRPhenomXPNR the choices of the higher-mode amplitudes (the order of the intermediate polynomials, the vetoes of collocation points). IMRPhenomXPNR's precession angles are continuous: LAL's switches between prescriptions, the number of SpinTaylor samples and where LAL's adaptive integration stops do not enter the model. The spins are the parameters that move the precession most, and they show the largest disagreements; for nearly aligned in-plane spins the waveform is also far from linear on the scale of the step.

One case is not a jump but a point where no derivative exists. After every step of its dynamics, NRSur7dq4 rescales each spin to its initial magnitude (LAL's `PrecessingNRSur_normalize_y`). For a body with no spin, an infinitesimal spin then evolves by a factor that depends on its direction, not only on its size. The waveform therefore has a cone-shaped kink in that body's three spin components: one-sided finite differences tend to different values, and no step size makes them converge. At such a point the JAX derivative is finite, as the transform tests require, but it is a convention rather than a derivative. The table reports these components apart. As soon as a body has any spin, even an aligned one, its derivatives agree with finite differences as for the other binaries.

IMRPhenomXPNR. For the full IMRPhenomXPNR waveform, the finite differences agree with the derivatives to 1.9×10^{-7} at worst. At the tag `lal-faithful` they disagreed by up to ~10^{-2}, most in the in-plane spins: LAL's finite differences in the intermediate region of the higher modes, with steps of 1e-9 and 1e-7, left rounding noise of ~1e-8 in the waveform that was not smooth in the parameters, and the SpinTaylor waveform jumped by LAL's integration error wherever the number of samples changed. The model takes exact derivatives and has no samples.

Cost of derivatives

FUNCTION	COMPILE	VALUE	JVP	COMPILE JACFWD	JACFWD	COMPILE GRAD	GRAD	COMPILE VMAP(GRAD)	VMAP(GRAD) / BINARY	GRAD VS J^TW
IMRPhenomXPNR	32.5 s	19.04 ms	107 ms	55.4 s	80.74 ms	54.0 s	90.46 ms	80.9 s	52.89 ms	2.4×10^{-15}
NRSur7dq4	2.9 s	3.03 ms	8.20 ms	6.1 s	20.92 ms	10.3 s	19.67 ms	12.3 s	16.25 ms	5.9×10^{-15}
simulate_td IMRPhenomXPNR (2,2)	6.7 s	5.62 ms	99.12 ms	24.7 s	112 ms	25.8 s	101 ms	37.7 s	80.52 ms	2.4×10^{-13}
simulate_fd NRSur7dq4	1.8 s	2.88 ms	7.51 ms	6.1 s	21.09 ms	10.6 s	20.20 ms	12.6 s	22.95 ms	6.3×10^{-15}

Times of the first call (which compiles) and of later calls, for IMRPhenomXPNR on 2048 frequencies, NRSur7dq4 on 2048 samples and the two conversions. `grad` is of a scalar, a weighted sum of the polarizations. `vmap(grad)` is over 8 binaries (4 for the conversions). The last column is the relative difference between `jax.grad` and the Jacobian contracted with the same weights.

For IMRPhenomXPNR a gradient costs 90.46 ms, 4.8× the cost of a waveform. It is a forward-mode Jacobian of twelve columns, evaluated together. Its compilation takes 54.0 s. A `jax.jvp` along a direction that moves every parameter costs as much as `jax.jacfwd`, because the rule computes the columns of all the parameters with non-zero tangents and then contracts them. When only some parameters are differentiated, as in `jax.grad` with respect to the masses alone with the rest of the binary held fixed, only their columns are computed. NRSur7dq4 is differentiated directly in reverse mode: a gradient costs 19.67 ms (6.5× a waveform) after 10.3 s of compilation.

13 Performance

All timings are on the CPU (Apple M2 Ultra, 24 cores), with JAX's default settings and LALSimulation single-threaded, except those of NRSur7dq4 in MLX (section 7). The binaries and grids are those of waveformrs's Python benchmark: IMRPhenomXPNR for 36 + 29 $M_\odot$ on 8033 frequencies (20–1024 Hz, $\Delta f = 1/8$ Hz), and NRSur7dq4 for 40 + 20 $M_\odot$ at 4096 Hz from 20 Hz. IMRPhenomXPNR has the parity fix; LAL 6.2.0 does not, at the same cost.

WAVEFORM	FIRST CALL	PUBLIC FUNCTION	COMPILED FUNCTION	WAVEFORMRS	LAL	LAL / COMPILED
IMRPhenomXPNR, 8033 frequencies	13.9 s	9.83 ms	7.81 ms	3.16 ms	13.82 ms	1.8×
NRSur7dq4, 4096 Hz from 20 Hz	2.6 s	3.49 ms	2.76 ms	1.01 ms	5.94 ms	2.2×
`simulate_td` of IMRPhenomXPNR, 4096 Hz from 20 Hz	13.6 s	17.94 ms	–	6.02 ms	13.29 ms	0.7×
`simulate_fd` of NRSur7dq4, up to 2048 Hz from 20 Hz	1.7 s	3.54 ms	–	1.19 ms	5.76 ms	1.6×

One waveform per call. "First call" includes LAL's checks, tracing and compilation. "Public function" is a later call of `wf.polarizations` (or `td_polarizations`, `simulate_*`), including the checks on the host and the copy of the result. "Compiled function" is the jitted pure function alone. waveformrs is its fastest path from Python. Medians of 7 repeats. Every call is for a different binary of a cycle of 16. Data: `report/data/timings.json`.

A compiled IMRPhenomXPNR waveform takes 7.81 ms, against 13.82 ms for LALSimulation and 6.60 ms (3.16 ms on its grid path) for waveformrs. The public function adds 1.62 ms of checks on the host: LAL's checks of the binary and a compiled pass through the set-up and the SpinTaylor integration, which reports a failed integration as an error; the pure function skips them. Every `jnp.where` evaluates both branches, loops whose length depends on the binary run as `while_loop`s, which XLA cannot fuse across iterations, and XLA's CPU code has no fused multiply-adds. XLA keeps about 4.2 cores busy for one IMRPhenomXPNR waveform, while LAL and waveformrs use one; JAX on a single core is still to be measured. NRSur7dq4 is dominated by vectorisable work (the fits at the nodes and the rotation of every mode sample), and takes 2.76 ms against 5.94 ms for LAL and 1.01 ms for waveformrs. The conversion `simulate_td` of IMRPhenomXPNR takes 17.94 ms, against 13.29 ms for LAL with multibanding off (11.94 ms with its default multibanding) and 6.02 ms for waveformrs.

One long waveform

For a light binary from a low frequency the cost is that of the long uniform grid of the frequencies. A precessing neutron star–black hole binary (10 + 1.4 $M_\odot$, spins (0.5, 0.2, 0.4) and (0.02, −0.01, 0.03)) up to 2048 Hz (`report/xpnr_long.py`):

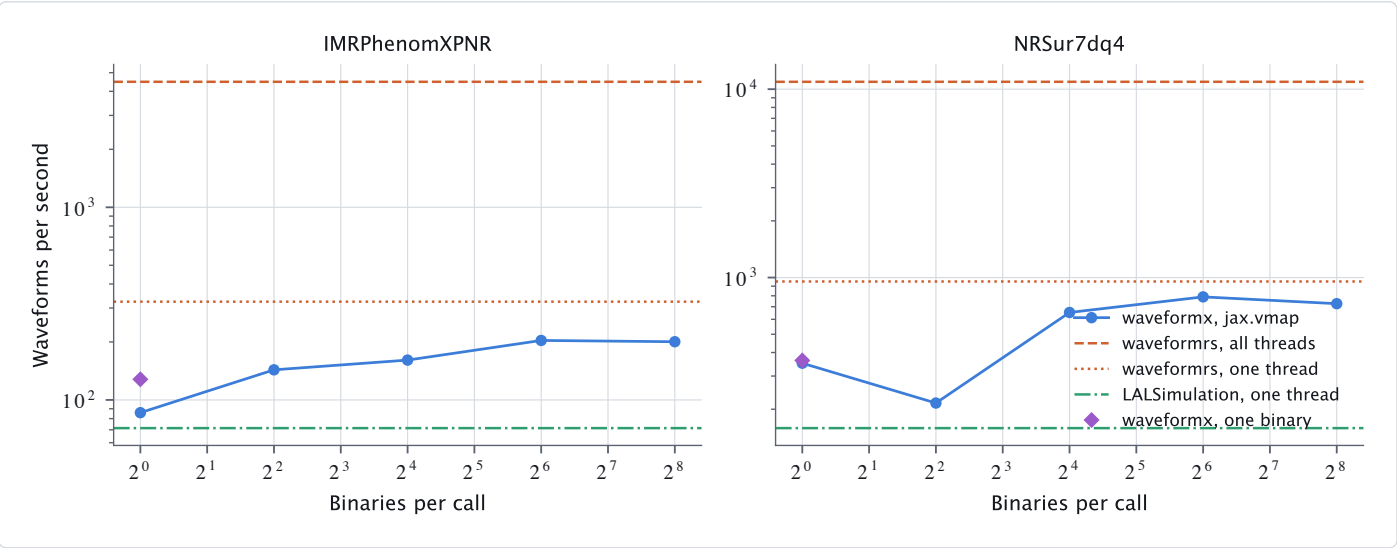
GRID	FREQUENCIES	COMPILED FUNCTION	PUBLIC FUNCTION	MLX (FLOAT32)	WAVEFORMRS (GRID PATH)	LAL (MULTIBANDING)
10 Hz, $\Delta f = 1/64$ Hz	130,433	30.95 ms	31.59 ms	70.04 ms	224 ms (173 ms)	547 ms (506 ms)
10 Hz, $\Delta f = 1/128$ Hz	260,865	43.97 ms	46.80 ms	71.86 ms	317 ms (221 ms)	731 ms (638 ms)
20 Hz, $\Delta f = 1/32$ Hz	64,897	17.60 ms	18.42 ms	40.38 ms	81.74 ms (59.00 ms)	187 ms (169 ms)

One waveform per call, $f_{ref} = f_{min}$; medians of 7 calls after a warm-up. LAL is `SimInspiralChooseFDWaveformSequence` at the frequencies (in parentheses `SimInspiralChooseFDWaveform`, which interpolates with its multibanding).

Before this, the same waveforms took 152 ms and 193 ms in JAX from 10 Hz and 86 ms from 20 Hz, and 156 ms and 217 ms in MLX. Three changes, none of which changes the waveform beyond a mismatch of 10^{-9}: the grid of the PNR interpolants (a few thousand nodes rather than 228,536, section 3), the PNR α of

the antisymmetric mode from those splines, and one binary's frequencies in one chunk. In MLX, LAL's moving average of the antisymmetric amplitude, an order-40 recurrence that a SIMD group stepped through one frequency at a time (124 ms at 260,865 frequencies), is solved on uniform grids for all the frequencies at once, by products with powers of its matrix (2 ms), and a single binary's integration no longer starts with room for 512 steps, which it overflowed and repeated. What remains in JAX is the SpinTaylor integration (12 ms for the 1,474 steps of this binary, ~8 µs each, the cost of XLA's thunks rather than of the arithmetic), the rest of the set-up (4 ms) and the angles, modes and twist at the frequencies (~28 ms over some hundred memory-bound kernels); in MLX the integration (61 ms: one SIMD group, ~40 µs per step).

Batches



Waveforms per second against the number of binaries per call of `jax.jit(jax.vmap(model))`, for binaries around the benchmark binary (masses ±10%, spin components in ±0.5, random angles). The horizontal lines are waveformrs's batch function and a loop over LALSimulation for 64 binaries.

BINARIES PER CALL	1	4	16	64	256
IMRPhenomXPNR, waveforms/s	86	143	161	204	201
IMRPhenomXPNR, compilation	16.8 s	16.4 s	17.2 s	17.8 s	19.9 s
IMRPhenomXPNR, cores busy	3.9	4.6	5.1	5.0	4.8
NRSur7dq4, waveforms/s	351	216	652	790	727
NRSur7dq4, compilation	2.4 s	2.2 s	2.1 s	2.2 s	2.7 s
NRSur7dq4, cores busy	5.5	4.3	4.4	6.9	6.9

Throughput, compilation time and cores kept busy (process CPU time over wall time) for each batch size. NRSur7dq4 is evaluated on LAL's output grid of the benchmark binary.

64 BINARIES	LAL LOOP	WAVEFORMRS, 1 THREAD	WAVEFORMRS, ALL THREADS	WAVEFORMX	FIRST CALL
IMRPhenomXPNR	71	324	4,483	158	19.9 s
NRSur7dq4	159	953	10,944	688	2.4 s

Waveforms per second through the public batch functions (`batch_polarizations`, `batch_td_polarizations`), including LAL's checks of every binary on the host.

Under `jax.vmap`, IMRPhenomXPNR reaches 204 waveforms per second. Batching amortises the per-call overhead and XLA spreads the vectorised work over cores, but the loops run to the longest binary in the batch and the branches of all binaries are computed. Each batch size is compiled separately (17.8 s for 64). waveformrs on all threads, with a thread per binary and no padding, remains faster on a CPU. The JAX version is meant for code that is itself written in JAX: a likelihood that is jitted and differentiated end to end, or a sampler that runs on an accelerator.

Memory

The arrays of a jitted function have fixed shapes, so IMRPhenomXPNR holds room for the longest case whatever the binary: the grid of the PNR interpolants has room for 524,288 points (a few thousand used for light binaries from 10 Hz; LAL's uniform grid for a binary neutron star from 10 Hz has about 290,000) and the SpinTaylor solution for 8,192 steps. A forward-mode derivative carries a copy of every array for each of the twelve parameters, and `jax.vmap` one for each binary, so that the batched Jacobian of 64 binaries needed 25.6 GB of temporary buffers (1.4 GB now). LAL's splines hold six arrays of it (α, β, γ and their second

derivatives). Three changes, none of which changes the waveform beyond rounding (mismatch below 1e-15, Jacobians within 1e-13): the SpinTaylor steps hold 44 numbers instead of 58 and are read in place; under `jax.vmap` the binaries run in blocks of 16 (4 for the Jacobian), each block vectorised, about as fast as the whole batch at once; and above 16,384 frequencies the angles, modes and twist run over chunks of the frequencies (LAL's moving average of the antisymmetric amplitude, which reads all of them, first). One binary's waveform itself, without derivatives, takes chunks of 524,288 frequencies: XLA's kernels on a chunk of 16,384 are slower per frequency, which cost a long waveform ~20 ms. The interpolants of the phase grid (`extended_phase_grid`) also hold two arrays fewer (the splines of γ and of the unwrapped α rather than of γ ± α). The memory no longer grows with the batch, nor with the frequencies beyond the outputs themselves. It does not depend on the masses or f_{min}: the shapes are the same for every binary. The MLX port sizes its arrays to each binary's grid and was already lean: 64 binaries on 4,096 frequencies take – of GPU memory, – with the Jacobian.

FUNCTION	BINARY	FREQUENCIES	BINARIES	TEMPORARIES BEFORE	TEMPORARIES NOW	RATIO	OUTPUT	PEAK RESIDENT
waveform	2.7 $M_\odot$ from 10 Hz	1,024	1	31 MB	28 MB	1.1×	0 MB	1.1 GB
waveform	2.7 $M_\odot$ from 10 Hz	16,384	1	32 MB	28 MB	1.1×	1 MB	1.0 GB
waveform	2.7 $M_\odot$ from 10 Hz	262,144	1	76 MB	76 MB	1.0×	8 MB	1.3 GB
`jax.jacfwd`	2.7 $M_\odot$ from 10 Hz	1,024	1	410 MB	360 MB	1.1×	0 MB	3.1 GB
`jax.jacfwd`	2.7 $M_\odot$ from 10 Hz	16,384	1	424 MB	366 MB	1.2×	6 MB	3.1 GB
`jax.jacfwd`	2.7 $M_\odot$ from 10 Hz	262,144	1	944 MB	534 MB	1.8×	96 MB	3.5 GB
`jax.vmap`	2.7 $M_\odot$ from 10 Hz	1,024	64	1.9 GB	442 MB	4.5×	2 MB	1.6 GB
`jax.vmap`	2.7 $M_\odot$ from 10 Hz	16,384	64	2.0 GB	458 MB	4.4×	32 MB	1.6 GB
`jax.vmap(jax.jacfwd)`	2.7 $M_\odot$ from 10 Hz	1,024	64	25.6 GB	1.4 GB	18.5×	24 MB	4.4 GB
`jax.vmap(jax.jacfwd)`	2.7 $M_\odot$ from 10 Hz	16,384	64	26.0 GB	1.4 GB	18.3×	384 MB	4.7 GB
waveform	60 $M_\odot$ from 20 Hz	16,384	1	32 MB	28 MB	1.1×	1 MB	989 MB
`jax.jacfwd`	60 $M_\odot$ from 20 Hz	16,384	1	424 MB	366 MB	1.2×	6 MB	3.0 GB

Memory of the compiled IMRPhenomXPNR, (2,2) mode, on uniform frequencies from f_{min} to 2048 Hz (or 0.3/M). "Temporaries" are the temporary buffers of XLA's compiled function (`memory_analysis()`), before (410d6f6) and with these changes; "Output" is the size of the result, "Peak resident" the largest resident memory of the process now, compilation and JAX itself (about 1 GB) included. Data: `report/data/xpnr_memory.json` (`report/xpnr_memory.py`).

14 Differences from LALSimulation

- **IMRPhenomXPNR: LAL's physics, not its numerics.** The SpinTaylor angles for every binary, solved once with a high-order integrator and evaluated from its dense output; LAL's discrete switches of the angles made continuous; exact derivatives in the higher modes; no emulation of LAL's arithmetic; the parity fix of lalsuite !2593 (section 3). The version that reproduced LAL's numerics step for step, with LAL's fallbacks and the released model without the parity fix as options, is commit `0bbd3c5` , tag `lal-faithful` .
- **Rounding, not bit-identity.** waveformrs reproduces LAL bit for bit on macOS arm64, using the fused multiply-adds that clang emits. JAX has no portable fused multiply-add and XLA reorders arithmetic, so NRSur7dq4 and the layers of IMRPhenomXPNR below its precession angles agree with LAL to rounding. Only on the host, where the conversions' lengths and epochs hinge on the last bits, are LAL's fused multiply-adds reproduced exactly.
- **Solar masses, not kilograms.** Every model, in both precisions, and the conversions' bounds on the duration compute with the masses in solar masses, times through `MTSUN_SI` and distances in Mpc, where LAL passes the masses through kilograms with G and c, which changes the results by rounding only.
- **The merger-ringdown continuation of the SpinTaylor angles.** LAL attaches the merger-ringdown α and β to the SpinTaylor angles with their values and derivatives at 0.97–1.0 of the end of the inspiral, which it reads from its splines near their last knots: the natural end condition puts the derivatives 0.6–5% off the exact ones. The model takes them from the SpinTaylor solution, exact, which moves the merger-ringdown angles; for heavy binaries and late references the waveforms differ from LAL's by mismatches of up to ~2×10^{-4}. This is an accepted difference: the model's values are the more accurate ones.
- **The zero crossings of the quintic intermediate amplitude.** LAL's `CrossZeroP5` , which decides whether a higher mode's intermediate amplitude drops from a fifth-order polynomial, reads shifted entries of GSL's packed roots. It misses some roots and depends on the order in which GSL finds them, which rounding can change. Here the signs of the quintic at 257 points decide (`hm/roots.py` ; at the tag, every root of GSL's companion matrix was tested with its own real and imaginary parts). Among the single-mode tests, one binary takes another polynomial for its (3,−3) mode, which then differs from LAL by a mismatch of 4.4e-7.
- **Values where LAL divides by zero.** At isolated points of the parameter space, LAL's MSA formulas divide by an exact zero and produce inf or NaN. Here the values there are finite, and so are the gradients nearby, for example at equal masses or for a single spin.
- **No multibanding.** LAL's `SimInspiralChooseFDWaveform` uses multibanding by default for IMRPhenomXPNR; waveformx, like waveformrs, evaluates every frequency. `SimInspiralChooseFDWaveformSequence` has multibanding off, and the conversions are compared with it off.
- **IMRPhenomD's quasi-normal-mode frequencies.** They come from closed-form fits in the final spin rather than LAL's spline through its table: they differ by about a part in 10^6 and change the waveform by mismatches of 10^{-9} or less (section 6).
- **No data race.** LAL's MSA and PNR angles race under OpenMP; its comparisons run single-threaded.
- **The scale of the output.** Both models return the strain times 10^{21}. `scaled_strain=False` , which `WaveformOptions.lalsimulation_defaults()` also sets, gives LAL's scale.

15 Limitations and future work

- **Six approximants.** Only TaylorF2, IMRPhenomD, IMRPhenomXAS, IMRPhenomXHM, IMRPhenomXPNR and NRSur7dq4 are public; IMRPhenomXP and XPHM with their own flags are not exposed. TaylorF2, IMRPhenomD and IMRPhenomXAS have their polarizations at given frequencies only, not their modes or LAL's uniform grids; none has the conversions, nor in MLX LAL's errors (they give NaN). TaylorF2 has LAL's default orders only, without tidal terms. IMRPhenomD in MLX computes each binary's table in float32, so heavy binaries lose more to float32 there than in JAX. IMRPhenomXHM has LAL's defaults only (no mode arrays, release versions or multibanding).
- **Compilation time.** IMRPhenomXPNR takes 15–20 s to compile, its gradient under a minute, and every array shape and batch size compiles separately. JAX's persistent compilation cache (`jax_compilation_cache_dir`) makes the cost one-off per machine. The tests use it; the package does not set it up.
- **Accelerators.** The JAX versions ran on the CPU only. In JAX, both models run in float32 only through their pure functions, since the public functions and the conversions are float64; Neither model has been measured on a GPU through JAX. NRSur7dq4 and IMRPhenomXPNR run on Apple GPUs through their MLX versions (section 7, section 3), which have limits of their own:
  - it is float32 only, since MLX has no float64 on the GPU;
  - they cannot be differentiated in forward mode (`mx.jvp`) or mapped with `mx.vmap` , which MLX does not carry through custom Metal kernels. They take batches themselves; NRSur7dq4's gradients are reverse mode, IMRPhenomXPNR's are its forward-mode Jacobian contracted in a custom VJP;
  - IMRPhenomXPNR in MLX has only the pure function at given frequencies (no LAL grids or conversions), depends on JAX to trace the model, and sizes its arrays with the binaries (the SpinTaylor steps, the grid of the PNR interpolants), so it reads sizes back from the GPU and does not run under `mx.compile` .
- **Forward-mode derivatives of IMRPhenomXPNR.** A gradient costs one Jacobian column per parameter with a non-zero tangent, evaluated together. That is affordable for the twelve parameters of a binary (section 12), but it does not scale to derivatives with respect to the frequencies or to many other inputs. A reverse-mode path would need the loops with traced lengths rewritten with fixed bounds.
- **Discontinuities.** IMRPhenomXPNR inherits the discrete choices of LAL's higher modes (versions of the amplitudes by mass ratio and spin, vetoes, the order of the intermediate polynomials, the second intermediate region of extreme mass ratios, the collocation points of the (3,3) phase by the sign of the total spin); gradient-based samplers near such a boundary see the derivative of one side only. At zero in-plane spin the waveform is continuous but has no derivative, since LAL's fits depend on the magnitude of the in-plane spin. Making them smooth would change LAL's model.
- **Long waveforms.** The conversions' Fourier transforms of light binaries from low frequencies hold millions of points, and the grid of the PNR interpolants keeps room for 524,288 nodes whatever the binary (a few thousand are used). These are fixed-shape arrays held in memory. Batches and frequencies are computed a part at a time (section 13), but the room of every binary computed together keeps its full size. `extended_phase_grid` differs from the default by mismatches up to ~2e-3 for binary neutron stars from 10 Hz (a near-constant offset of the angles above f_{ref}).
- **Multibanding.** LAL's multibanding is not implemented (as in waveformrs).

16 Reproducing the results

```
# environment (conda-forge), with waveformrs checked out next to waveformx
mamba create -n waveformx -c conda-forge python=3.12 python-lalsimulation jax numpy h5py scipy matplotlib pytest
conda activate waveformx
pip install -e .
(cd ../waveformrs/python && maturin develop --release)       # optional: the waveformrs comparisons
pip install -e ".[mlx]"                                        # optional: NRSur7dq4 and IMRPhenomXPNR in MLX (Apple silicon)

# the tests (the data file NRSur7dq4_v1.0.h5 from zenodo.org/records/14999310,
# found through NRSUR7DQ4_DATA, LAL_DATA_PATH or ../waveformrs/lal_data)
WAVEFORMX_SLOW=1 python -m pytest

# optional: LALSimulation 6.2.0 + lalsuite !2593 (else waveformrs stands in for it)
CC=/path/to/conda-forge/clang-19 bash ../waveformrs/scripts/build_patched_lalsimulation.sh ../lalsuite build/lal-2593
export PATCHED_LALSIMULATION=$PWD/build/lal-2593/liblalsimulation.37.dylib

# every measurement and this report, on an idle machine (MLX_PYTHON: a Python
# with MLX, if not the same)
bash report/measure_all.sh

# or one at a time
python report/validate_xpnr.py --n-draws 200
python report/validate_nrsur7dq4.py --n-draws 200
python report/float32_nrsur7dq4.py
python report/mlx_nrsur7dq4.py                               # needs MLX
python report/mlx_xpnr.py                                    # needs MLX
python report/xas.py                                         # the MLX parts need MLX
python report/xhm.py                                         # the MLX parts need MLX
python report/taylorf2_phenomd.py --model TaylorF2           # the MLX parts need MLX
python report/taylorf2_phenomd.py --model IMRPhenomD
python scripts/fit_phenomd_qnm.py                            # IMRPhenomD's QNM fits, from LALSuite's sources
python -m pytest tests/test_nrsur7dq4_mlx.py tests/test_xpnr_mlx.py tests/test_xas_mlx.py tests/test_xhm_mlx.py tests/test_taylorf2_mlx.py
tests/test_phenomd_mlx.py --junitxml=report/data/pytest_mlx.xml
python report/validate_conversions.py
python report/gradients.py
python report/benchmark.py
python report/xpnr_cost.py
python report/reference_xpnr.py
python report/crossmodel.py                                  # the benchmark set: --select (once)
python report/build_report.py
```

17 Commit history

The commits of this branch, oldest first. The package was built model by model and milestone by milestone, each checked against LAL before the next. The conversions and the speed-ups of IMRPhenomXPNR were developed in parallel on two branches and merged. Commits that only add or regenerate this report are not listed. The repository has no remote.

> **Two bugs found while preparing an earlier version of this report, both fixed.** First, the conversions and the IMRPhenomXPNR speed-ups had been developed on separate branches. After the merge, `jax.grad` of `simulate_td_function` failed with "Linearization failed to produce known values". The conversions called the model through a reverse-mode wrapper, while the speed-ups had made the model's loops forward-mode only. The conversions now use the model's forward-mode Jacobians, with respect to the binary and to the starting frequency, which itself depends on the binary (`deeffce`). Second, when the first use of NRSur7dq4's data was the pure function under `jax.jit`, the cached data were built inside the trace and leaked tracers into later calls. The tests had always read the data through a plain call first; the gradient measurements of section 12 did not (`0a93b55`, with a test).

COMMIT	TYPE	SUMMARY
229d095	FEAT	the waveformjax package: the waveformrs interface over JAX models
7494344	DOC	per-model transform tests in AGENTS.md
48c6d70	FEAT	NRSur7dq4 data reader, parameter-space fits and GSL interpolation in JAX
839cf04	FEAT	NRSur7dq4 dynamics as fixed-size JAX loops
4da7038	FEAT	NRSur7dq4 coorbital waveform, frame rotation and polarizations
cf2462d	FEAT	NRSur7dq4 MODEL: LAL's checks, output grid, modes and batches
4b1fead	TST	find the NRSur7dq4 data next to a parent directory (git worktrees)
c296de4	TST	NRSur7dq4 against LALSimulation 6.2.0 and under jit, vmap and grad
20502f6	PERF	NRSur7dq4 splines with GSL's factorised systems, cached output grids

COMMIT	TYPE	SUMMARY
a303ecb	DOC	NRSur7dq4 in the README and its original code in NOTICE.md
9d7cb40	DOC	NRSur7dq4 dynamics docstring; share |ra|^2a |rb|^2b in the rotation
f5f9745	PERF	NRSur7dq4 dynamics steps as a few vector operations
d0acb33	MAINT	drop an unused table of the NRSur7dq4 waveform module
0c084c5	FEAT	IMRPhenomXAS (2,2) mode in JAX, with the fits generated from waveformrs
ea84f6e	FEAT	PNR fits and angle helpers, and the SpinTaylor evolution in JAX
91fe045	FEAT	IMRPhenomXHM release 122019 higher modes in JAX
2fcc136	FEAT	MSA (223), NNLO (102) and SpinTaylor (330) precession angles in JAX
8761b4d	FEAT	IMRPhenomXPNR polarizations in JAX
c7f6cba	FIX	one optimization barrier per value
59e311b	FEAT	IMRPhenomXPNR J-frame modes, validation and transform tests
0b05193	TEST	one switch, WAVEFORMJAX_SLOW=1, for the random-draw tests of both models
afe668f	TEST	the NRSur7dq4 random-draw test requires most draws compared, for any NRSUR7DQ4_DRAWS
f57a940	FEAT	LAL's conditioning in JAX: duration bounds, epochs, tapers and the Butterworth filter
787c98e	FEAT	simulate_td and simulate_fd: LAL's SimInspiralTD and SimInspiralFD
a182f49	DOC	the conversions in the README, and their original code in NOTICE.md
4e5a7ae	TEST	the conversions against SimInspiralTD and SimInspiralFD
c9e8eba	DOC	the XPNR adapter's values depend on the whole frequency array
9bdcf41	DOC	the accuracy of the conversions, as measured
568227a	TEST	the conversions as pure functions under jit, vmap and grad
545046e	PERF	SpinTaylor derivatives with the 3-vectors as arrays
eb49f8b	PERF	GSL's quintic solver in loops that end where GSL's do
4fe9828	PERF	IMRPhenomXPNR derivatives in forward mode, batched under jax.vmap
a2d6a68	PERF	the intermediate polynomials of the higher modes chosen together
8e042ff	PERF	one branch for PNR versions 102 and 223; smaller SpinTaylor code
b5c0df1	PERF	only the PNR beta of the binary's region; alpha of the antisymmetric part where used
864c255	PERF	batch-aware branches and loop bodies traced under jax.jit
4476314		Merge the IMRPhenomXPNR compile, batching and forward-mode speed-ups
deeffce	FIX	the conversions of IMRPhenomXPNR differentiate the model in forward mode
0a93b55	FIX	NRSur7dq4 data read inside jax.jit are concrete arrays
027e282	FEAT	IMRPhenomXPNR without fallbacks, and extended, continuous SpinTaylor angles
c31b9c3	FEAT	NRSur7dq4 in float32, with the strain times 1e21 by default
366535e	FEAT	smoother extended XPNR angles where LAL samples a step or a branch
321ebea	DOC	the smoothness study of the extended XPNR model (measurement and draft section)
26c400e	FIX	finite gradients of the extended XPNR at aligned and vanishing spins
aa32207	FEAT	NRSur7dq4 in MLX (float32, Apple GPUs), as a separate module
157fd2e	FIX	the smooth XPNR angles where L is close to -J
d78a40c	FEAT	the MLX NRSur7dq4 dynamics in one Metal kernel, with its Jacobian
7632c48	PERF	the MLX NRSur7dq4 rotation to the inertial frame in Metal kernels
a315b13	PERF	tune the MLX NRSur7dq4 dynamics kernel
87c01bb	FEAT	the MLX NRSur7dq4 on LAL's grid, in batches and conditioned, with its report section
aa460c3	FEAT	IMRPhenomXPNR compiles only the SpinTaylor angles by default, direct extended angles, quick tests
6994cac	PERF	evaluate the SpinTaylor splines of the XPNR set-up at all its points at once
630477e	PERF	one copy of the SpinTaylor derivatives per RKF45 step, materialized mode arrays
3dfe8cf	PERF	the PNR beta variants of the interpolation grid computed once each

COMMIT	TYPE	SUMMARY
4c32137	PERF	the carry of the XPNR set-up scan starts dependent on the binary
4008e75	PERF	h_+ and h_x of a twisted-up mode computed in one kernel
5f97d83	FIX	CrossZeroP5 tests every root of the quintic; QR steps on whole rows
eeec783	PERF	the double QR sweep of the quintic's roots in a loop over k
a6ce002	PERF	plain fused multiply-adds in the PNR beta connection
1df3534	PERF	the intermediate polynomials of the higher modes from a linear solve
6123358	PERF	the (2,2) coefficients of the antisymmetric mode reused by the symmetric one
a87a5c0	PERF	the SpinTaylor alpha of the antisymmetric mode evaluated with the PNR grid
471262c	PERF	the coefficients of the (2,1), (3,3) and (4,4) modes in one vmapped body
12eef25	PERF	the (3,2) mode in the vmapped body of the higher modes
a2994a6	PERF	batched branches merge into one output rather than selecting among zeros
1aaa72e	PERF	SpinTaylor loop bodies small enough to run on XLA's calling thread
3e07c6a	PERF	the antisymmetric alpha of the grid loop only below its transition
d253bf0	TST	JAX's persistent compilation cache for the tests
09c88fd	PERF	the cross products of the SpinTaylor equations from gathers
ac3b193	PERF	the moving average of the antisymmetric amplitude ratio in one solve
3ff80dc	PERF	one copy of the stopping condition in the smooth end of the fine run
2b1e132	BUG	finite derivatives of the evolved spins at aligned spins
59cc4cc		ENH: IMRPhenomXPNR in float32, and the strain times 1e21
f634a68		ENH: the lean IMRPhenomXPNR (WaveformOptions(lean=True))
3c42b28	PERF	lean IMRPhenomXPNR in one pass, beta connection evaluated once
7d871df	PERF	lean quintic zero-crossing test by signs; exact HM derivatives at one point
38a70a3	DOC	the lean IMRPhenomXPNR in the report
b7b6028	TST	the lean IMRPhenomXPNR in stages, the reference of its MLX port
90372b6		ENH: jaxprs of the lean model's stages as scalar programs and Metal kernels
78c4b0e		ENH: the set-up of the lean IMRPhenomXPNR as generated MLX kernels
7183d19		ENH: the lean SpinTaylor solution as Metal kernels
bdb42c9		ENH: the SpinTaylor angles at points and the PNR interpolants in MLX
5fdeedb		ENH: the lean IMRPhenomXPNR in MLX: polarizations, Jacobian and mx.grad
07994df	BUG	finite float32 derivatives of the IMRPhenomXPNR higher modes
d74e287	PERF	the MLX lean IMRPhenomXPNR: SIMD lanes per component, packed stages
fe9389f	PERF	the MLX lean IMRPhenomXPNR: tables read by index, batches in chunks
4c33e19	DOC	the lean IMRPhenomXPNR in MLX in the report and the README
cbdb398	PERF	the MLX lean IMRPhenomXPNR: skip the padded grid points of the PNR angles
080fbbd	DOC	the timings of the MLX lean IMRPhenomXPNR after skipping padded grid points
f7155e2	BUG	finite reverse-mode derivatives of the peak of the ringdown amplitude
1462cb8		ENH: IMRPhenomXAS in JAX and MLX, from a table of coefficients per binary
cc67e42	DOC	IMRPhenomXAS in the report
14c424f		ENH: a reference frequency scaled by the mass for the lean IMRPhenomXPNR
d05bc15	DOC	the mass-scaled reference frequency in the report and the README
12fb663	MAINT	ignore working plans and .DS_Store
9c90114		ENH: the lean IMRPhenomXPNR with the mass-scaled reference does not depend on the reference
e62b628	DOC	the reference-invariant lean IMRPhenomXPNR in the report and the README
641a50b		ENH: IMRPhenomXHM in JAX and MLX, with its modes
f1ac969	DOC	IMRPhenomXHM in the report

COMMIT	TYPE	SUMMARY
63ede06	DOC	the IMRPhenomXHM measurements and the rebuilt report
0bbd3c5	MAINT	rename the package to waveformx
5031a34		ENH: TaylorF2 in JAX
20d1de0		ENH: closed-form fits of IMRPhenomD's quasi-normal-mode frequencies
9cf2c4a		ENH: IMRPhenomD in JAX
c2f56a0		ENH: matches maximised over time, phase and polarisation
2668270	BUG	finite derivatives of IMRPhenomD's peak frequency where gamma2 > 1
d8d1964		ENH: TaylorF2 and IMRPhenomD in MLX
5fc958d		ENH: IMRPhenomD's table in float64 for float32 binaries in JAX
378233d	DOC	TaylorF2 and IMRPhenomD in the README and NOTICE
26e08b5	DOC	TaylorF2 and IMRPhenomD in the report
c3e8ca2	DOC	the TaylorF2 and IMRPhenomD measurements and the rebuilt report
35c51cc	TST	TaylorF2 and IMRPhenomD among the approximants
a2e2b36		ENH: a fixed set of binaries for IMRPhenomXPNR against NRSur7dq4
6abad63		ENH: reference_mf checked per model
2315e2d		ENH: the mass-scaled reference for TaylorF2 and IMRPhenomD
54b674a	DOC	the mass-scaled reference of TaylorF2 and IMRPhenomD
dc08b3c	MAINT	IMRPhenomXPNR keeps LAL's physics, not its numerics
1afb1c8	DOC	README and report scripts for the single IMRPhenomXPNR
b159045	DOC	IMRPhenomXPNR against NRSur7dq4 in the report
a07f6f6	DOC	the IMRPhenomXPNR-NRSur7dq4 measurements and the rebuilt report
eb2786f		Merge branch 'refs/heads/claude/taylorf2-phenomd-947cb5' into xpnr-spintaylor-only
b968af9		Merge the cross-model matches into xpnr-spintaylor-only
c04a27a	MAINT	the comparison with NRSur7dq4 for the single IMRPhenomXPNR
6bf41bc	MAINT	IMRPhenomXPNR in solar masses without mass_unit and g_si
b178c38	MAINT	NRSur7dq4's set-up in solar masses
486d508	MAINT	the conversions' duration bounds in solar masses
a7bfa83	DOC	solar masses, not kilograms, among the differences from LAL
2df397b		ENH: the mass-scaled reference for IMRPhenomXAS and IMRPhenomXHM
2e7ddf4	TST	one compiled function per model in the quick tier, in parallel
acac0ed	TST	a float32 floor for the MLX IMRPhenomXPNR with the mass-scaled reference
7cb0ddd	PERF	two arrays of the PNR grid fewer in IMRPhenomXPNR
0d0d587	PERF	a lighter record of the SpinTaylor steps
aa187f7	PERF	IMRPhenomXPNR computes the binaries of a vmap in blocks
11e8842		ENH: the mass-scaled reference for NRSur7dq4
fa4bc82		ENH: one pair of reference converters for every model; README and report
bfe6402	TEST	the tests and the MLX stand-in solution follow the lighter interpolants and record
2810b05	PERF	IMRPhenomXPNR evaluates long frequency grids in chunks
c098c92	TST	IMRPhenomXAS and IMRPhenomXHM take reference_mf
8289859	BUG	IMRPhenomXPNR continuous as the in-plane spins vanish
505ef97	BUG	IMRPhenomXPNR continuous at equal masses
cc1100b	DOC	IMRPhenomXPNR through vanishing spins and equal masses
25eb91e	TEST	frequency chunks and Jacobian blocks give the values and derivatives of one pass
0a70acc	DOC	the memory of IMRPhenomXPNR in the report
bb435e2	BUG	LAL's splines of the PNR angles

COMMIT	TYPE	SUMMARY
f11a424		Merge claude/pnr-grid-alpha into xpnr-spintaylor-only
06a2066		Merge claude/crossmodel-0e6e13 into xpnr-spintaylor-only
fbbc6f5		Merge claude/smooth-spin-cd2e35 into xpnr-spintaylor-only
6471dab		Merge claude/no-si-units-189b2f into xpnr-spintaylor-only
cc84672		Merge claude/massref-fb3f75 into xpnr-spintaylor-only
410d6f6		Merge claude/quick-tests-2e36e5 into xpnr-spintaylor-only
7a62d55	DOC	the exact angles at the merger-ringdown connection, an accepted difference from LAL
c4fe621		Merge claude/pnr-grid-alpha (the accepted difference at the merger-ringdown connection)
02acd97		Keep the traced precision per thread
20e6399		Merge claude/awesome-hofstadter-4e7d0f (the traced precision per thread)
ead5be8		Merge xpnr-spintaylor-only (410d6f6) into claude/spintaylor-memory-02d214
3372647		Merge branch 'xpnr-spintaylor-only' into claude/spintaylor-memory-02d214
7e8cf5f		Merge claude/spintaylor-memory-02d214 (less memory for IMRPhenomXPNR)
0e9809a		ENH: IMRPhenomXPNR_NNLO and IMRPhenomXPNR_MSA, the PNR angles on the NNLO and MSA angles
1330bd9		ENH: IMRPhenomXPNR_NNLO and IMRPhenomXPNR_MSA in MLX; README and report
e972d4d	PERF	MSA angles once per grid point, sn skips unneeded AGM steps; FIX: NNLO phase grid
5ad4c24		Merge claude/xpnr-prescriptions-313b89 (IMRPhenomXPNR_NNLO and IMRPhenomXPNR_MSA)
118ea39		ENH: one long IMRPhenomXPNR waveform fast
0318e4b		Merge 5ad4c24 (IMRPhenomXPNR_NNLO and IMRPhenomXPNR_MSA) into claude/xpnr-long-fast-d177a9
6f4c58d	TST	the NNLO angles of three heavy binaries, resolved by the node grid where LAL's grid is not